



UNIVERSITÄT
LEIPZIG

Wirtschaftswissenschaftliche Fakultät

Institut für Wirtschaftsinformatik

Professur Informationsmanagement

Prof. Dr. Bogdan Franczyk

Nico Heider

Thema

*Autonomes Landen von Drohnen in Agrarlandschaften basierend auf
Deep-Reinforcement-Learning*

Bachelorarbeit zur Erlangung des akademischen Grades

Bachelor of Science – *Informatik*

vorgelegt von: *Fritz Körner*

Matrikelnummer: *3748012*

E-Mail-Adresse: *fk67rahe@studserv.uni-leipzig.de*

Telefonnummer: *+4915221457367*

Anschrift: *Klarastraße 17
04229 Leipzig*

Leipzig, den 15. Juni 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	I
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Formelverzeichnis	VI
Abkürzungsverzeichnis	VII
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielstellung	2
1.3 Aufbau der Arbeit	2
2 Fachliche Grundlagen	4
2.1 Reinforcement Learning	4
2.1.1 Markov Decision Process	4
2.2 Strategieoptimierung	5
2.2.1 Policy-Gradient-Methoden	5
2.2.2 Proximal Policy Optimization	6
2.3 Convolutional Neural Networks	7
2.4 Achsenparallele Begrenzungsboxen	8
3 Stand der Forschung	9
3.1 Autonomes Landen von Drohnen	9
3.2 Tiefenbildbasierte Hindernisvermeidung	11
3.3 Reinforcement Learning für Drohnensteuerung	12
4 Methodik	16
4.1 Systemüberblick	16
4.2 Formulierung als Markov-Entscheidungsprozess	16
4.2.1 Zustandsraum	17
4.2.2 Aktionsraum	17

4.2.3	Übergangsmodell	18
4.2.4	Belohnungsfunktion	18
4.2.5	Terminierung	19
4.3	Netzwerkarchitektur	19
4.4	Algorithmus	20
4.4.1	Proximal Policy Optimization	20
4.4.2	Aktionsverteilung und Tanh-Squashing	20
4.4.3	Hyperparameter	21
4.5	Trainingsumgebung	21
4.5.1	Physiksimulator	21
4.5.2	Drohnenmodell	21
4.5.3	Korridorgeometrie	22
4.5.4	Hindernisse	22
4.6	Kaskadierender PID-Regler	23
4.7	Trainingsprozess	23
4.8	Evaluation	24
4.8.1	Evaluationsmetriken	24
4.8.2	Versuchsreihe 1: Generalisierung auf unbekannte Hindernisformen	24
4.8.3	Versuchsreihe 2: Transfer in eine realistische Agrarumgebung . . .	25
5	Ergebnisse	27
5.1	Trainingsverhalten	27
5.2	Phase 1: Korridornavigation	28
5.2.1	Nachuntersuchung: Oberflächenbasierte Kollisionserkennung . . .	29
5.3	Phase 2: Weinberg-Transfer	30
5.3.1	Sensitivitätsanalyse des Erfolgskriteriums	31
5.4	Vergleich RL vs. RRT-Connect	31
6	Diskussion	32
6.1	Leistungsfähigkeit in der Trainingsumgebung	32
6.1.1	Trainingsverhalten und Konvergenz	32
6.1.2	Dominante Fehlerart und Flugverhalten	33
6.2	Transferleistung auf den Weinberg	33
6.2.1	Leistungsabfall und Verschiebung der Fehlerarten	33

6.2.2	Einordnung als Sim-to-Sim Transfer	34
6.2.3	Perceptual Aliasing als Erklärung für die Terrain-Kollisionen	34
6.3	Leistungslücke zum Pfadplaner	35
6.4	Limitationen	36
6.4.1	Trainingssetup	36
6.4.2	Sensorik und Architektur	36
6.4.3	Umgebungsdiversität	36
7	Fazit und Ausblick	38
7.1	Beantwortung der Forschungsfragen	38
7.2	Perspektiven für zukünftige Forschung	39
	Zusammenfassung	40
	Literatur	41
	Anhang	IX
	Offenlegung der Benutzung von generativer KI	XX
	Erklärung	XXI

Abbildungsverzeichnis

2.1	Agent-Umgebung-Interaktion im MDP	4
2.2	PPO-Clipping-Funktion	6
4.1	Systemarchitektur	16
4.2	Netzwerkarchitektur	20
4.3	Simuliertes Drohnenmodell	22
4.4	Korridorgeometrie	22
4.5	Simulationsansicht des Korridors	23
4.6	Photogrammetrisches Weinberg-Mesh	25
4.7	Zielpositionen in der Weinbergumgebung	26
5.1	Belohnungsverlauf über 6 Seeds	27
5.2	Episodenergebnisse Phase 1	29
5.3	Simulationsansicht erfolgreicher und gescheiterter Episode	29
5.4	Episodenergebnisse Phase 2	30
6.1	Perceptual Aliasing im Tiefenbild	35

Tabellenverzeichnis

3.1	Vergleich lernbasierter Drohnennavigationssysteme	14
4.1	Komponenten des Zustandsvektors	17
4.2	Gewichte der Belohnungskomponenten	19
4.3	Hyperparameter des PPO-Algorithmus.	21
4.4	Konfiguration des Trainingsprozesses.	24
5.1	Konvergenzergebnis der 6 Trainingsseeds	28
5.2	Checkpoint-Selektion	28
5.3	Vergleich AABB- und oberflächenbasierte Kollisionserkennung	30
5.4	Sensitivitätsanalyse des Erfolgsradius in Phase 2	31
5.5	Vergleich RL-Agent und RRT-Connect	31

Formelverzeichnis

2.1	Übergangsdynamik des MDP	4
2.2	Diskontierter Ertrag	5
2.3	Zustandswertfunktion	5
2.4	PPO-Clipping-Zielfunktion	6
2.5	Vollständige PPO-Verlustfunktion	7
4.1	Zusammensetzung des Zustandsraums	17
4.2	Zustandsvektor	17
4.3	Normalisierung des Tiefenbilds	17
4.4	Aktionsraum	18
4.5	Berechnung der Sollposition	18
4.6	Belohnungsfunktion	18
4.7	Hindernisproximität	19
4.8	Tanh-Squashing der Aktionen	21
4.9	Log-Wahrscheinlichkeit unter Tanh-Transformation	21
4.10	Pfadeffizienz	24

Abkürzungsverzeichnis

AABB	Axis-Aligned Bounding Box
CNN	Convolutional Neural Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ELU	Exponential Linear Unit
EMA	Exponential Moving Average
GAE	Generalized Advantage Estimation
GNC	Guidance, Navigation and Control
GNSS	Global Navigation Satellite System
GPU	Graphics Processing Unit
GSO	Google Scanned Objects
HPC	High-Performance Computing
IMU	Inertial Measurement Unit
IQR	Interquartile Range
KL	Kullback-Leibler
MDP	Markov Decision Process
MLP	Multilayer Perceptron
PID	Proportional-Integral-Derivative
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
RRT	Rapidly-exploring Random Tree
RTK	Real Time Kinematic
SAC	Soft Actor-Critic
SLAM	Simultaneous Localization and Mapping
TD3	Twin Delayed Deep Deterministic Policy Gradient
TRPO	Trust Region Policy Optimization
URDF	Unified Robot Description Format
VRAM	Video Random Access Memory

1 Einleitung

1.1 Motivation und Problemstellung

Die Landwirtschaft steht vor wachsenden Herausforderungen: Der Klimawandel führt zu erhöhtem Schädlingsdruck, veränderten Niederschlagsmustern und sinkenden Erträgen [1, 2, 3], während eine wachsende Weltbevölkerung und zunehmender Fachkräftemangel den Produktionsdruck zusätzlich verschärfen [4, 5, 6]. Im Rahmen der digitalen Landwirtschaft bieten Drohnen (unbemannte Luftfahrzeuge) vielversprechende Lösungsansätze [7]: Sie ermöglichen unter anderem die gezielte Ausbringung von Pflanzenschutzmitteln sowie großflächiges Crop-Monitoring mittels Remote Sensing [8, 9] und können so dazu beitragen, die Landwirtschaft effizienter und resilienter zu gestalten [10, 11].

Trotz umfangreicher Forschung zu Drohnensystemen bleibt autonomes Landen eine offene Herausforderung [12]. Unfallfreies Landen ist für die meisten Flugmissionen essentiell; gleichzeitig erfordern die vielfältigen Landeszenarien jeweils angepasste Lösungen. Bisherige Ansätze reichen von Global Navigation Satellite System (GNSS)-gestützter Positionierung [13] über markerbasierte visuelle Verfahren [14] bis hin zu markerfreien Systemen, die Landezonen eigenständig aus Sensordaten ableiten [15]. Dabei zeigt sich eine wiederkehrende Einschränkung: Systeme zur Hindernisvermeidung navigieren erfolgreich durch komplexe Umgebungen, ohne gezielt zu landen [16, 17], während Landesysteme überwiegend in hindernisfreien Szenarien operieren [18, 19]. Die Kombination beider Teilaufgaben, Präzisionslandung mit gleichzeitiger Hindernisvermeidung, ist insbesondere in natürlichen Umgebungen mit dichter Vegetation bislang kaum untersucht. Weinberge sind ein konkretes Beispiel solcher Umgebungen: Reihenabstände von typischerweise 1,5 bis 2 m und dichtes Blattwerk erzeugen beengte Platzverhältnisse, die eine autonome Landung zwischen den Reihen erheblich erschweren.

Reinforcement Learning (RL) hat sich in jüngerer Zeit als wirksame Lösungsstrategie für Drohnenaufgaben etabliert [20, 21, 12]: Ein RL-Agent erlernt eine Steuerungsstrategie durch Interaktion mit einer simulierten Umgebung, ohne dass ein explizites Umgebungsmodell oder manuell definierte Steuerungsregeln benötigt werden. In Kombination mit *Convolutional Neural Networks* (CNNs) können aufgabenrelevante Merkmale direkt aus Sensordaten extrahiert werden [22], was eine bildbasierte Hindernisvermeidung ohne manuelle Merkmalsdefinition ermöglicht.

Da Drohnen engen Gewichts- und Energiebeschränkungen unterliegen [23], beschränkt sich die vorliegende Arbeit auf eine minimale Sensorkonfiguration aus Tiefenkamera und propriozeptivem Zustandsvektor und setzt auf einen lernbasierten Ansatz, um diese begrenzte Sensorik für die autonome Landung in hindernisreichen Umgebungen auszunutzen.

1.2 Zielstellung

Ziel der Arbeit ist die Entwicklung eines *Reinforcement-Learning*-basierten Verfahrens, das eine autonome Multirotor-Drohne befähigt, mithilfe lokaler Tiefensensorik in einer hindernisreichen Umgebung sicher zu landen. Das System kombiniert einen propriozeptiven Zustandsvektor mit tiefenbildbasierter Umgebungswahrnehmung, um Hindernisse während des Landevorgangs zu erkennen und ihnen auszuweichen. Die Leistungsfähigkeit des trainierten Agenten wird anhand einer abstrakten Trainingsumgebung evaluiert und anschließend im *Zero-Shot-Transfer*, also ohne erneutes Training in der Zielumgebung, auf eine Umgebung mit realistischer Vegetationsstruktur getestet.

Dadurch ergeben sich folgende Fragestellungen:

Hauptfragestellung

Inwiefern kann ein *Reinforcement-Learning*-basierter Ansatz eine autonome Drohne befähigen, mithilfe lokaler Tiefensensorik in hindernisreichen Umgebungen sicher zu landen?

Nebenfragestellungen

1. Wie leistungsfähig ist der trainierte Agent in der Trainingsumgebung, und welche Faktoren begrenzen die Erfolgsrate?
2. In welchem Maß lässt sich die erlernte Navigationsleistung auf eine geometrisch verschiedenartige Umgebung mit realistischer Vegetationsstruktur übertragen?
3. Welche Leistungslücke besteht zwischen einem lernbasierten Agenten mit lokaler Sensorik und einem Pfadplaner mit vollständiger Umgebungsinformation?

1.3 Aufbau der Arbeit

Kapitel 2 führt die für diese Arbeit relevanten Grundlagen ein: die theoretischen Grundlagen des *Reinforcement Learning* von der MDP-Formulierung über Wertfunktionen bis hin zu *Proximal Policy Optimization*, *Convolutional Neural Networks* zur Merkmalsextraktion aus Bilddaten sowie achsenparallele *Bounding Boxes* zur Kollisionserkennung. Kapitel 3 gibt einen Überblick über bestehende Ansätze zur autonomen Drohnenlandung und Hinder-

nisvermeidung und identifiziert die Forschungslücke, die diese Arbeit adressiert. Kapitel 4 beschreibt das entwickelte System, einschließlich der MDP-Formulierung, der CNN-MLP-Architektur, der simulierten Trainingsumgebung und des zweiphasigen Evaluationsprotokolls. Kapitel 5 präsentiert die experimentellen Ergebnisse beider Evaluationsphasen sowie den Vergleich mit einem klassischen Pfadplaner. Kapitel 6 interpretiert die Ergebnisse im Kontext der Forschungsfragen und diskutiert Limitationen des Ansatzes. Kapitel 7 fasst die zentralen Erkenntnisse zusammen und skizziert Perspektiven für weiterführende Arbeiten.

2 Fachliche Grundlagen

Dieses Kapitel führt die theoretischen Grundlagen ein, auf denen das entwickelte System aufbaut.

2.1 Reinforcement Learning

Reinforcement Learning bezeichnet das Lernen aus Interaktionen mit einer Umgebung, um ein Ziel zu erreichen. Ein Agent wählt in aufeinanderfolgenden Situationen Aktionen und erhält dafür ein skalares Belohnungssignal. Aus diesen Erfahrungen muss er eigenständig ableiten, welche Aktionen langfristig den höchsten Ertrag erzielen. Sofern nicht anders angegeben, folgt die Darstellung in diesem Abschnitt Sutton und Barto [24].

2.1.1 Markov Decision Process

Ein *Markov Decision Process* (MDP) formalisiert *Reinforcement Learning* Probleme als Interaktion zwischen einem Agenten und einer Umgebung in diskreten Zeitschritten $t = 0, 1, 2, \dots$. Der Agent beobachtet einen Zustand $S_t \in \mathcal{S}$, wählt eine Aktion $A_t \in \mathcal{A}$ und erhält eine Belohnung $R_{t+1} \in \mathcal{R}$ sowie einen neuen Zustand S_{t+1} (Abbildung 2.1).

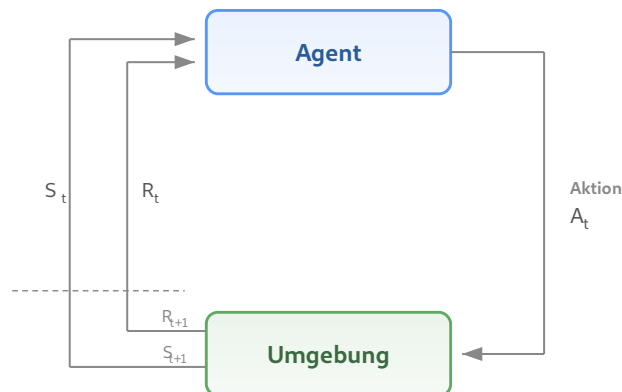


Abbildung 2.1.: Interaktion zwischen Agent und Umgebung in einem MDP (nach [24], S. 48).

Die Übergangsdynamik

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \quad (2.1)$$

charakterisiert die Umgebung vollständig. Wesentlich ist die Markov-Eigenschaft: Folgezustand und Belohnung hängen ausschließlich vom gegenwärtigen Zustand-Aktions-Paar ab.

Das Ziel des Agenten ist eine optimale Strategie π_* , die die kumulierte Belohnung maximiert.

2.2 Strategieoptimierung

Welche Verfahren zur Bestimmung einer optimalen Strategie geeignet sind, hängt maßgeblich von der Formulierung des *Reinforcement-Learning*-Problems ab. Die klassische Theorie liefert exakte Lösungen ausschließlich für endliche MDPs mit vollständig bekanntem Übergangsmodell; ihre Grundideen lassen sich jedoch auf komplexere Problemklassen verallgemeinern.

2.2.1 Policy-Gradient-Methoden

Die kumulierte Belohnung wird formal als erwarteter diskontierter Ertrag

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.2)$$

mit Diskontierungsfaktor $\gamma \in [0, 1)$ ausgedrückt. Die Güte einer Strategie wird durch die Zustandswertfunktion

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] \quad (2.3)$$

charakterisiert. Insbesondere bei hochdimensionalen Zustandsräumen und kontinuierlichen Aktionsräumen, wie sie in der Drohnensteuerung auftreten, stoßen klassische Verfahren an ihre Grenzen.

Policy-Gradient-Methoden umgehen diese Beschränkungen, indem sie die Strategie direkt als $\pi(a \mid s, \theta)$ parametrisieren und durch Gradientenaufstieg optimieren. Dies ermöglicht sowohl die Verwendung neuronaler Netze als Funktionsapproximatoren als auch die direkte Handhabung kontinuierlicher Aktionsräume, wie sie in der Drohnensteuerung auftreten. In der Praxis wird dies als *Actor-Critic*-Architektur umgesetzt: Ein Actor $\pi(a \mid s, \theta)$ repräsentiert die Strategie und wählt Aktionen; ein Critic $\hat{v}(s, \mathbf{w})$ schätzt die Zustandswertfunktion und bewertet die gewählten Aktionen. Der Critic dient dabei als varianzreduzierendes Referenzniveau für den Policy-Gradienten, da der Agent nicht mehr auf den verrauschten Episodenenertrag G_t angewiesen ist, sondern die gelernte Wertschätzung als stabilere Grundlage nutzt.

2.2.2 Proximal Policy Optimization

Policy-Gradient-Verfahren können durch zu große Strategieänderungen pro Optimierungsschritt destabilisiert werden, da die unter der alten Strategie gesammelten Erfahrungsdaten das Verhalten der aktualisierten Strategie nicht mehr abbilden [25]. Trust-Region-Methoden wie *Trust Region Policy Optimization* (TRPO) [26] begrenzen daher die Schrittweite über eine Kullback-Leibler-Divergenz-Schranke (KL) zwischen alter und neuer Strategie, erfordern dafür jedoch Approximationen zweiter Ordnung. *Proximal Policy Optimization* (PPO) [25] erzielt eine vergleichbare Stabilisierung allein durch eine geklippte Zielfunktion:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (2.4)$$

wobei $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{alt}}}(a_t | s_t)$ das Wahrscheinlichkeitsverhältnis zwischen neuer und alter Strategie ist, $\hat{A}_t$ der geschätzte Vorteil, berechnet mittels *Generalized Advantage Estimation* (GAE) [27], und ε die Clipping-Grenze. Das Clipping bewirkt *pessimistische* Updates: Verbessert eine Strategieänderung das Objective, wird der Anreiz bei $r_t > 1 + \varepsilon$ gekappt; verschlechtert sie es, greift die Minimumbildung und die Korrektur bleibt uneingeschränkt (Abbildung 2.2). So werden zu große Schritte in Richtung guter Erfahrungen gebremst, während schlechte Aktionen voll korrigiert werden.

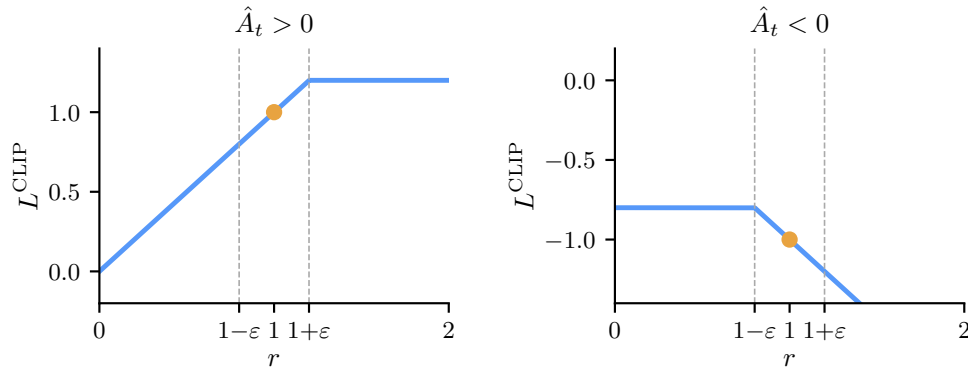


Abbildung 2.2.: L^{CLIP} als Funktion des Wahrscheinlichkeitsverhältnisses r . Links ($\hat{A}_t > 0$): der Anreiz wird bei $1 + \varepsilon$ gekappt. Rechts ($\hat{A}_t < 0$): die Korrektur bleibt uneingeschränkt (nach [25], Fig. 1).

Die vollständige Verlustfunktion von PPO kombiniert das Clipping mit zwei weiteren Termen:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta](s_t), \quad (2.5)$$

wobei $L^{VF} = (V_{\theta}(s_t) - V_t^{\text{targ}})^2$ der quadratische Fehler der Wertfunktionsschätzung des Critics ist und $H[\pi_{\theta}]$ einen Entropie-Bonus bezeichnet, der vorzeitige Konvergenz der Aktionsverteilung verhindert.

2.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) sind eine Klasse neuronaler Netze, die speziell für die Verarbeitung von Daten mit räumlicher Struktur entwickelt wurden, etwa Bilder in Form zweidimensionaler Pixelarrays. Sofern nicht anders angegeben, folgt die Darstellung in diesem Abschnitt LeCun u. a. [28].

Die Architektur eines CNN besteht aus einer Kaskade von Faltungs- und Pooling-Schichten. In einer Faltungsschicht sind die Einheiten in sogenannten *Feature Maps* organisiert. Jede Einheit ist nur mit einem lokalen Ausschnitt der vorherigen Schicht verbunden und verrechnet diesen mit einem Filter (auch Kernel) [29]. Alle Einheiten innerhalb einer *Feature Map* teilen denselben Filter. Verschiedene *Feature Maps* innerhalb einer Schicht verwenden unterschiedliche Filter und erkennen dadurch unterschiedliche lokale Muster. Die Filtergewichte werden nicht manuell festgelegt, sondern während des Trainings gelernt [29]. Die gemeinsame Nutzung der Gewichte beruht auf zwei Eigenschaften natürlicher Bilddaten: Benachbarte Werte bilden häufig korrelierte lokale Muster, und diese Muster treten unabhängig von ihrer Position im Bild auf.

Pooling-Schichten fassen die Aktivierungen benachbarter Einheiten zusammen, typischerweise durch Auswahl des Maximums innerhalb eines lokalen Fensters (*Max-Pooling*) [29]. Dies reduziert die räumliche Auflösung der Repräsentation und erzeugt eine gewisse Invarianz gegenüber kleinen Verschiebungen im Eingabebild.

Durch das Stapeln mehrerer Faltungs- und Pooling-Stufen entsteht eine hierarchische Merkmalsextraktion: Frühe Schichten erkennen einfache lokale Strukturen wie Kanten, mittlere Schichten kombinieren diese zu komplexeren Motiven, und tiefe Schichten repräsentieren ganze Objektteile. Die resultierende kompakte Repräsentation kann als Eingabe für nachgeschaltete Netzwerkkomponenten dienen. Mnih u. a. [22] zeigten erstmals, dass ein CNN in dieser Rolle als Merkmalsextraktor innerhalb eines *Reinforcement-Learning*-Systems eingesetzt werden kann, indem sie eine Strategie direkt aus hochdimensionalen Bilddaten lernten.

2.4 Achsenparallele Begrenzungsboxen

Eine achsenparallele Begrenzungsbox (*Axis-Aligned Bounding Box*, AABB) ist das kleinste achsenparallele Quader, das ein gegebenes Objekt vollständig umschließt. Daher wird jede tatsächliche Kollision mit dem Objekt auch als Kollision mit der AABB erkannt. Im Umkehrschluss kann ein Punkt jedoch innerhalb der AABB liegen, ohne das eingeschlossene Objekt zu berühren. Der Approximationsfehler wächst mit der Abweichung der Objektgeometrie von einer quaderförmigen Gestalt; bei konkaven oder stark unregelmäßigen Formen kann die AABB ein erheblich größeres Volumen als das Objekt selbst einnehmen.

Für die Kollisionserkennung in Echtzeitsystemen bietet die AABB einen günstigen Kompromiss: Der Abstandstest reduziert sich auf sechs skalare Vergleiche pro Achse und ist damit wesentlich effizienter als Distanzberechnungen auf der tatsächlichen Objektgeometrie. Dem steht die beschriebene konservative Approximation gegenüber, die zu falsch-positiven Kollisionserkennungen führen kann.

3 Stand der Forschung

3.1 Autonomes Landen von Drohnen

Bisherige Arbeiten zum autonomen Landen von Drohnen lassen sich in zwei grundlegende Kategorien einteilen: traditionelle und bildbasierte Methoden. Traditionelle Methoden stützen sich auf Signale externer Infrastruktur wie GNSS-Satelliten oder Funkbaken, um Position und Lage der Drohne zu bestimmen. Bildbasierte Methoden hingegen nutzen Kameradaten zur Wahrnehmung der Umgebung und lassen sich weiter unterteilen in markerbasierte Systeme, die definierte visuelle Referenzpunkte zur Lokalisierung verwenden, und markerfreie Systeme, die adäquate Landezonen eigenständig aus Bilddaten ableiten und somit keine zusätzliche Bodeninfrastruktur erfordern.

Orthogonal zur Sensorik lassen sich Ansätze danach unterscheiden, welche Teilaufgaben des Guidance, Navigation and Control (GNC) [30] sie adressieren und wie diese zusammenwirken. Modulare Methoden zerlegen das Problem in getrennte Komponenten wie Hinderniserkennung, Lagebestimmung oder Pfadplanung, die als aufeinanderfolgende Stufen einer Pipeline gelöst werden [30, 31]. *End-to-End*-Methoden hingegen bilden Sensorinput direkt auf Steuerkommandos ab, ohne solche expliziten Zwischenstufen [31].

GNSS ist die verbreitetste Methode zur Positionsbestimmung von Drohnen [32], erreicht in der Praxis jedoch nur Meter-Genauigkeit: Patrik u. a. [13] berichten von einer Landegenauigkeit von 1,11 m, und auch beim Open-Source-Autopiloten PX4 beträgt die Genauigkeit mehrere Meter [33, 14]. Real-Time-Kinematic-Korrekturen (RTK) können die Genauigkeit unter idealen Bedingungen auf Zentimeter-Niveau steigern [34]. Dass autonomes Landen bei bekannter Relativposition grundsätzlich beherrschbar ist, zeigen Voos und Bou-Ammar [35]; die zentrale Herausforderung liegt in der Positionsgenauigkeit, nicht im Regelungsentwurf. Für Präzisionslandung zwischen Rebzeilen (Reihenabstand 1,5–2 m) reicht GNSS allein nicht aus; zudem liefert es keinerlei Umgebungswahrnehmung.

Bildbasierte Landesysteme mit vordefinierten Referenzmarkern bieten eine deutlich höhere Präzision als rein GNSS-gestützte Methoden. So erreicht das auf Augmented Reality University of Cordoba (ArUco) basierende System von Wubben u. a. [14] durch deterministische Bildverarbeitung eine durchschnittliche Landeabweichung von 11 cm bei zuverlässiger Markerdetektion aus bis zu 30 m Höhe. Deterministische Ansätze setzen jedoch voraus, dass die Markererkennung unter allen Betriebsbedingungen robust funktioniert; bei Verdeckung,

wechselnden Lichtverhältnissen oder beschädigten Markern ist diese Annahme nicht gegeben.

RL bietet hier einen konzeptionellen Vorteil: Statt auf regelbasierte Detektionsalgorithmen angewiesen zu sein, lernt ein RL-Agent eine Steuerungsstrategie direkt aus Bilddaten. Polvara u. a. [18] demonstrieren diesen Ansatz erstmals für die autonome Landung: Ihr System nutzt sequentielle Deep-Q-Networks mit diskretem Aktionsraum und einer monokularen Graustufenkamera. Ausschließlich in Simulation mit *Domain Randomization* trainiert, erzielt das System Erfolgsraten von 89 % für den vertikalen Abstieg, die im realen Flug auf 61 % sinken. Eine zentrale Einschränkung bleibt der diskrete Aktionsraum, der die Steuerungspräzision limitiert.

Ladosz u. a. [19] erweitern den Ansatz auf bewegliche Landeplattformen und adressieren diese Einschränkung durch einen kontinuierlichen Aktionsraum, der erstmals auch die Vertikalgeschwindigkeit einschließt, während bei früheren Arbeiten der vertikale Abstieg typischerweise regelbasiert gesteuert wird. Der Agent erhält neben Graustufenbildern auch Daten einer inertialen Messeinheit (IMU; Geschwindigkeit und Lage) als Zustandsinformation. In der Simulation erreicht das System Erfolgsraten von bis zu 97,4 % bei einer Landeabweichung von 0,52 m; im realen Transfer sinkt die Rate auf rund 70 %. Die Autoren zeigen zudem, dass *Domain Randomization* zwar notwendig für den *Sim-to-Real*-Transfer ist, eine zu starke Randomisierung die Performance jedoch signifikant verschlechtert. Die Experimente beschränken sich auf einfache Szenarien ohne Hindernisse und gleichmäßig beleuchtete Innenräume.

Markerbasierte Systeme setzen jedoch vorplatzierte Infrastruktur voraus, und beide RL-Ansätze wurden ohne Hindernisse trainiert, wobei der *Sim-to-Real*-Transfer mit deutlichen Einbußen einhergeht [vgl. 18, 19]. Markerfreie Verfahren umgehen diese Einschränkung, indem sie Landezonen eigenständig aus Bilddaten ableiten [36, 15].

Yang u. a. [36] verfolgen einen modularen Ansatz, bei dem mithilfe eines monokularen Simultaneous Localization and Mapping (SLAM) Systems sichere Landezonen identifiziert werden. Das System demonstriert Landungen ohne GNSS, weist jedoch aufgrund der Pipeline-Komplexität eine Landezeit von rund zwei Minuten auf.

Bartolomei u. a. [15] wählen einen *End-to-End*-Ansatz: Ihr RL-Agent bildet Tiefenbilder und semantische Segmentierungen direkt auf diskrete Steuerkommandos ab und erreicht in der Simulation Erfolgsraten von über 70 % mit lediglich einer monokularen RGB-Kamera. Im Realflug landete der Agent erfolgreich, während sowohl eine kommerzielle als auch eine

State-of-the-Art-Lösung am verrauschten Sensorinput scheiterten. Einschränkend setzt das System voraus, dass semantische Segmentierungen und Tiefenbilder stets verfügbar sind, und der diskrete Aktionsraum limitiert die Steuerungsflexibilität.

Dass *Deep Reinforcement Learning* (DRL) auch unter dynamischen Umgebungsbedingungen effektive Landesteuerungen ermöglicht, zeigen Peter u. a. [37] mit TornadoDrone: Der TD3-Agent (*Twin Delayed Deep Deterministic Policy Gradient*) leitet Windeinflüsse nicht aus direkten Kraftmessungen, sondern aus indirekten Zustandsänderungen (Position, Geschwindigkeit) ab und übertrifft eine konventionelle PID-Regler-Baseline (Proportional-Integral-Derivative) bei Landungen unter Windturbulenzen deutlich. Die realen Experimente stützen sich jedoch auf ein Vicon-Lokalisierungssystem, das unter Einsatzbedingungen nicht verfügbar ist.

3.2 Tiefenbildbasierte Hindernisvermeidung

Loquercio u. a. [16] demonstrieren Hochgeschwindigkeitsnavigation durch komplexe Umgebungen mit bordeigener Sensorik. Das System nutzt Tiefenbilder einer Stereokamera und IMU-Daten, um kollisionsfreie Trajektorien zu erzeugen, die ein PID-Regler ausführt. Trainiert wird in Simulation mittels *Privileged Learning* mit einfachen Hindernisgeometrien; durch simuliertes Sensorrauschen gelingt ein *Zero-Shot*-Transfer in reale Umgebungen (wie dichte Wälder). Der Ansatz reduziert die Ausfallrate gegenüber modularen Planern um den Faktor 10, definiert Erfolg jedoch bereits bei 5 m Zielabstand.

Kim u. a. [38] schätzen Tiefenbilder aus monokularen RGB-Aufnahmen und treffen auf dieser Basis diskrete Ausweichentscheidungen mittels Dueling-Double-DQN (Deep Q-Network). Das System durchfliegt nach *Zero-Shot*-Transfer enge Korridore kollisionsfrei, beschränkt sich jedoch auf strukturierte Innenräume und enthält keine zielgerichtete Navigation.

Xu u. a. [17] nutzen direkte Tiefenbilder, um mittels PPO eine kontinuierliche Strategie für sichere Navigation in Umgebungen mit statischen und dynamischen Hindernissen zu trainieren. Die Architektur verarbeitet die Tiefenbilder zu einer 3D-Belegungskarte, die zusammen mit Drohnenzustandsdaten (Position, Geschwindigkeit, Orientierung) als Eingabe für ein *Actor-Critic*-Netzwerk dient. Nach *Zero-Shot*-Transfer in eine reale Umgebung mit dynamischen Hindernissen erzielt das System die geringste Kollisionsrate im Vergleich zu bestehenden Methoden. Auch hier beschränken sich die Testumgebungen auf strukturierte Indoor-Szenarien.

Gemeinsam zeigen diese Arbeiten, dass tiefenbildbasierte lernende Systeme Hindernisse zuverlässig vermeiden können, unabhängig davon ob die Tiefeninformation direkt gemessen [16, 17] oder aus RGB-Bildern geschätzt wird [38]. Allerdings evaluiert keine der genannten Arbeiten in landwirtschaftlichen Umgebungen wie Rebzeilen oder Obstplantagen.

Vereinzelte Arbeiten adressieren die Navigation in landwirtschaftlichen Umgebungen gezielt. Wei u. a. [39] demonstrieren ein lernbasiertes Drohnensystem für die autonome Navigation durch Obstplantagen-Reihen: Ein mittels *Imitation Learning* trainierter Controller steuert eine Multirotor-Drohne auf Basis einer front-montierten RGB-Kamera kollisionsfrei zwischen Baumreihen. Das System generalisiert auf ungesehene Umgebungen und kommt ohne GPS aus, beschränkt sich jedoch auf horizontale Reihennavigation. Für Weinberge zeigen Martini u. a. [40], dass ein DRL-Agent auf Basis von Tiefenbildern ohne Positionsinformation durch simulierte Rebzeilen navigieren kann, allerdings mit einem Bodenroboter statt einer Drohne. Beide Arbeiten bestätigen, dass Vegetation in landwirtschaftlichen Umgebungen eine eigenständige Navigationsherausforderung darstellt, die durch lernbasierte Ansätze mit minimaler Sensorik adressiert werden kann. Keine der Arbeiten behandelt jedoch die vertikale Landung zwischen Vegetationsstrukturen, die eine kombinierte Hindernisvermeidung und Präzisionspositionierung erfordert.

3.3 Reinforcement Learning für Drohnensteuerung

Über die Wahl der Sensorik hinaus beeinflusst die Steuerungsarchitektur die Leistungsfähigkeit lernbasierter Drohnensysteme maßgeblich. Alle in den Abschnitten 3.1 und 3.2 betrachteten lernbasierten Systeme folgen einem gemeinsamen Grundmuster: Ein gelerntes Modell bildet Sensorinput auf Steuerkommandos ab, die ein nachgeschalteter Flugregler ausführt. Die Systeme unterscheiden sich jedoch in der Abstraktionsebene dieser Ausgabe: von diskreten Bewegungsprimitiven [18, 38, 15] über kontinuierliche Geschwindigkeitskommandos [19] bis hin zu Kurzzeittrajektorien, die ein expliziter PID-Regler abfliegt [16].

Die Wahl der Abstraktionsebene beeinflusst die Lernperformance substantiell: Eßer u. a. [41] zeigen in einer systematischen Studie über verschiedene Roboterplattformen und Aufgaben, dass der optimale Aktionsraum stark aufgabenabhängig ist und unterschiedliche Abstraktionsebenen zu deutlich verschiedenem Explorationsverhalten und finaler Performance führen. Speziell für Multirotor-Drohnen vergleichen Kaufmann u. a. [42] drei Abstraktionsebenen: Geschwindigkeitskommandos, kollektiven Schub mit Drehraten sowie direkte Einzelmotor-schübe. Die mittlere Ebene (Schub und Drehraten) erweist sich als beste Balance zwischen

Agilität und Transferrobustheit; die höchste Abstraktion (Geschwindigkeit) schränkt die Manövrierfähigkeit ein, während die niedrigste (Einzelmotorschub) zu latenzempfindlich für den Realtransfer ist.

Eng mit der Wahl des Aktionsraums verknüpft ist die Wahl des Trainingsalgorithmus. Als *Policy-Gradient*-Verfahren eignet sich PPO (vgl. Abschnitt 2.2.2) besonders für kontinuierliche Aktionsräume und wird in der Drohnensteuerung häufig eingesetzt. Da RL grundsätzlich große Datenmengen benötigt, hat sich GPU-parallele Simulation als effektiver Ansatz etabliert: Rudin u. a. [43] zeigen, dass massiv-parallele Simulation mit hunderten bis tausenden Umgebungsinstanzen die Trainingszeit drastisch reduziert und schnellere Konvergenz ermöglicht. PPO eignet sich als *On-Policy*-Verfahren für dieses Paradigma, da es die parallel erzeugten Daten direkt verarbeitet und keinen *Replay Buffer* benötigt. Obwohl *Off-Policy*-Verfahren wie *Soft Actor-Critic* (SAC) eine höhere Sample-Effizienz bieten, zeigen Tayar u. a. [44], dass PPO bei sicherheitskritischen Präzisionsaufgaben in beengten Räumen zuverlässiger konvergiert als SAC. In den in diesem Kapitel betrachteten Arbeiten setzen Bartolomei u. a. [15], Xu u. a. [17] und Kaufmann u. a. [42] PPO ein; Kaufmann u. a. [20] demonstrieren den bislang eindrucksvollsten Einsatz, bei dem ein ausschließlich in Simulation trainiertes System drei menschliche Weltmeister im physischen Drohnenrennen schlägt.

Die vorliegende Arbeit wählt eine noch höhere Abstraktionsebene als die von Kaufmann u. a. [42] untersuchten: Der RL-Agent gibt eine Zielposition vor, die ein kaskadierender PID-Regler (Position $\rightarrow$ Geschwindigkeit $\rightarrow$ Lage $\rightarrow$ Drehzahl) in Motorkommandos umsetzt. Diese Wahl opfert bewusst Agilität zugunsten reduzierter Lernkomplexität, da die Aufgabe Präzisionslandung statt agilen Flugs erfordert. Die Ausgabe des Agenten bleibt interpretierbar, der PID-Regler kann unabhängig vom RL-Training auf die Zielplattform abgestimmt werden, und die Lernkomplexität reduziert sich auf die Frage, *wohin* die Drohne fliegen soll.

Die Wahl der Sensorausstattung ist ein weiterer zentraler Designparameter: Drohnen sind aufgrund ihrer Energie- und Gewichtsbeschränkungen typischerweise nur mit minimaler Sensorik ausgestattet, bestehend aus Kameras und Distanzsensoren [15]. Semerikov u. a. [23] zeigen in ihrer Übersichtsarbeit, dass Vision-Inertial-Kombinationen eine gute Balance zwischen Wahrnehmungsleistung und Systemkomplexität bieten, während leistungsfähigere Sensorsetups entsprechend größere Drohnen mit höherer Lade- und Batteriekapazität erfordern. Gleichzeitig ermöglichen moderne Algorithmen, insbesondere lernbasierte Ansätze, dass auch mit leichtgewichtiger Sensorik anspruchsvolle Aufgaben gelöst werden können, die bisher komplexere Hardware voraussetzten. Die betrachteten Arbeiten bestätigen diesen

Trend: Erfolgreiche Navigation gelingt mit einer einzelnen Kamera [38, 18, 15] oder Tiefenkamera mit Zustandsdaten [16, 17]. Die vorliegende Arbeit folgt diesem Prinzip: Als Eingabe dienen eine Tiefenkamera und ein propriozeptiver Zustandsvektor.

Tabelle 3.1 stellt die in diesem Kapitel betrachteten Arbeiten anhand ihrer Kernmerkmale gegenüber.

Tabelle 3.1.: Vergleich lernbasierter Systeme zur autonomen Drohnennavigation und -landung. Die Spalten *Hind.* und *Landung* kennzeichnen, ob das System aktive Hindernisvermeidung bzw. Präzisionslandung adressiert.

Arbeit	Sensorik	Methode	Aktionsraum	Hind.	Umgebung	Landung
Polvara u. a. [18]	Mono RGB	SDQN	diskret	–	Indoor	✓
Ladosz u. a. [19]	Mono RGB, IMU	DrQv2	kont.	–	Indoor	✓
Yang u. a. [36]	Mono RGB, Barometer	SLAM	–	✓	Outdoor	✓
Bartolomei u. a. [15]	Tiefe, Semantik	PPO	diskret	✓	Outdoor	✓
Peter u. a. [37]	IMU	TD3	kont.	–	Indoor	✓
Loquercio u. a. [16]	Stereo Tiefe, IMU	IL	kont.	✓	Outdoor	–
Kim u. a. [38]	Mono RGB, Tiefe	D3QN	diskret	✓	Indoor/ Outdoor	–
Xu u. a. [17]	Mono RGB, Tiefe, Zustand	PPO	kont.	✓	Indoor	–
Wei u. a. [39]	Mono RGB	IL	kont.	✓	Agrar	–
Martini u. a. [40] ^a	Tiefe, Zustand	SAC	kont.	✓	Agrar	–
Eigene Arbeit	Tiefe, Zustand	PPO	kont.	✓	Agrar	✓

^a Bodenroboter (UGV). Zustand = propriozeptiver Zustandsvektor (Position, Orientierung, Geschwindigkeit).

Zusammenfassend zeigt der Stand der Forschung zwei zentrale Lücken: Erstens werden Hindernisvermeidung und Präzisionslandung überwiegend getrennt adressiert (vgl. Tabelle 3.1): Systeme zur Hindernisvermeidung navigieren durch komplexe Umgebungen, ohne gezielt zu landen [16, 38, 17], während Landesysteme überwiegend in hindernisfreien Szenarien operieren [18, 19, 37]. Amendola u. a. [12] bestätigen dieses Muster in ihrer Übersichtsarbeit: Zahlreiche RL-basierte Landesysteme reduzieren das Problem auf zwei Dimensionen mit festem vertikalem Abstieg, was die Konvergenz beschleunigt, jedoch die Strategie einschränkt und komplexere Verhaltensweisen wie Hindernisvermeidung verhindert. Zweitens fehlen Evaluationen in landwirtschaftlichen Umgebungen: Alle betrachteten Systeme testen in Innenräumen, Wald- oder abstrakten Outdoor-Umgebungen; die spezifischen Herausforderungen von Agrarumgebungen mit engen Vegetationsstrukturen bleiben unberücksichtigt. Zudem bleibt der *Sim-to-Real*-Transfer eine offene Herausforderung, insbesondere bei Nutzung visueller Sensordaten [12]: Die Erfolgsraten sinken systematisch beim Übergang in die reale Welt (von 89 % auf 61 % [18], von 97,4 % auf rund 70 % [19]).

Keine der betrachteten Arbeiten kombiniert Präzisionslandung, aktive Hindernisvermeidung und tiefenbildbasierte Wahrnehmung in einer Agrarumgebung (vgl. Tabelle 3.1). Die vorliegende Arbeit adressiert diese Lücken: Sie untersucht, ob ein RL-Agent mit kontinuierlichem

Aktionsraum und Tiefenbildern in Simulation lernen kann, in beengten Umgebungen mit Hindernissen autonom zu landen. Ein vollständiger *Sim-to-Real*-Transfer liegt außerhalb des Rahmens dieser Arbeit. Um die Transferfähigkeit der gelernten Strategie dennoch zu untersuchen, wird ein *Zero-Shot*-Transfer in eine geometrisch verschiedenartige Simulationsumgebung (*Sim-to-Sim*) eingesetzt.

4 Methodik

Dieses Kapitel beschreibt das entwickelte System zur autonomen Drohnenlandung mit Hindernisvermeidung. Ein RL-Agent trifft Navigationsentscheidungen auf hoher Abstraktions-ebene, während ein kaskadierender PID-Regler diese in Motorkommandos umsetzt. Training und Evaluation finden vollständig in Simulation statt. Ein *Sim-to-Real*-Transfer liegt außerhalb des Rahmens dieser Arbeit.

4.1 Systemüberblick

Das System gliedert sich in zwei Steuerungsebenen (Abbildung 4.1): Auf der oberen Ebene trifft ein RL-Agent alle 3,0 s eine Navigationsentscheidung. Auf der unteren Ebene setzt ein kaskadierender PID-Regler diese mit 100 Hz in Motordrehzahlen um. Das Entscheidungsintervall stellt sicher, dass die Drohne zwischen zwei Entscheidungen zur Ruhe kommt und das Tiefenbild aus einer stabilen Fluglage aufgenommen wird.

Der Agent erhält einen normalisierten Zustandsvektor $\mathbf{o}_t \in \mathbb{R}^{17}$ und ein Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ einer vertikal nach unten gerichteten Kamera mit 90° Sichtfeld. Ein geteilter CNN-Encoder extrahiert Hindernismerkmale aus dem Tiefenbild. Der resultierende Merkmalsvektor wird mit dem Zustandsvektor konkateniert und an die MLP-Köpfe (Multilayer Perceptron) von Actor und Critic weitergegeben. Der Actor sampelt eine vierdimensionale Aktion $\mathbf{a}_t \in [-1, 1]^4$, die der PID-Regler über 300 Simulationsschritte in Motordrehzahlen umsetzt. Der Agent muss somit ausschließlich lernen, *wohin* die Drohne fliegen soll.



Abbildung 4.1.: Architektur des vorgestellten Systems: Der RL-Agent erhält Tiefenbild und Zustandsvektor, sampelt eine Aktion und gibt Sollwerte vor. Der PID-Regler setzt diese über in Motordrehzahlen um.

4.2 Formulierung als Markov-Entscheidungsprozess

Die autonome Drohnenlandung wird als endlicher, episodischer MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ formalisiert (vgl. Abschnitt 2.1.1), mit Diskontierungsfaktor $\gamma = 0,99$.

4.2.1 Zustandsraum

Der Zustand $s_t \in \mathcal{S}$ setzt sich aus einem numerischen Zustandsvektor $\mathbf{o}_t \in \mathbb{R}^{17}$ und einem Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ zusammen:

$$s_t = (\mathbf{o}_t, \mathbf{D}_t). \quad (4.1)$$

Der Zustandsvektor

$$\mathbf{o}_t = (\tilde{\mathbf{p}}_{\text{rel},t}, \mathbf{q}_t, \tilde{\mathbf{v}}_t, \tilde{\boldsymbol{\omega}}_t, \mathbf{a}_{t-1}) \quad (4.2)$$

enthält die in Tabelle 4.1 aufgeschlüsselten Größen. Die Tilde kennzeichnet skalierte und auf $[-1, 1]$ geklippte Werte.

Tabelle 4.1.: Komponenten des Zustandsvektors $\mathbf{o}_t \in \mathbb{R}^{17}$.

Größe	Dimension	Beschreibung	Skalierung
$\tilde{\mathbf{p}}_{\text{rel},t}$	3	Relative Position zum Ziel	$\times \frac{1}{13}, \in [-1, 1]$
$\mathbf{q}_t$	4	Orientierungsquaternion (w, x, y, z)	—
$\tilde{\mathbf{v}}_t$	3	Lineargeschwindigkeit	$\times 0,4, \in [-1, 1]$
$\tilde{\boldsymbol{\omega}}_t$	3	Winkelgeschwindigkeit	$\times \frac{1}{\pi}, \in [-1, 1]$
$\mathbf{a}_{t-1}$	4	Aktion des vorherigen Zeitschritts	—

Eine am Drohnenkörper vertikal nach unten gerichtete Kamera liefert ein Tiefenbild $\mathbf{d}_{\text{roh}}$, welches auf $[0, 1]$ normalisiert wird:

$$\mathbf{D}_t = \min\left(\max\left(\frac{\mathbf{d}_{\text{roh}}}{d_{\text{max}}}, 0\right), 1\right), \quad d_{\text{max}} = 20 \text{ m}. \quad (4.3)$$

Das Tiefenbild liefert die einzige Information über Hindernisse; der Zustandsvektor enthält keine explizite Hindernisrepräsentation.

4.2.2 Aktionsraum

Der Aktionsraum ist vierdimensional und kontinuierlich:

$$\mathbf{a}_t = (a_x, a_y, a_z, a_\psi) \in [-1, 1]^4. \quad (4.4)$$

Die ersten drei Komponenten definieren einen Positionsversatz relativ zur aktuellen Drohnenposition $\mathbf{p}_t$ und ergeben die Sollposition $\mathbf{p}_{\text{soll}}$:

$$\mathbf{p}_{\text{soll}} = \mathbf{p}_t + \begin{pmatrix} a_x \cdot \sigma_x \\ a_y \cdot \sigma_y \\ a_z \cdot \sigma_z \end{pmatrix}, \quad \boldsymbol{\sigma} = (1,0; 1,0; 2,0) \text{ m.} \quad (4.5)$$

Die erhöhte z -Skala ermöglicht größere vertikale Schritte für den vorwiegend vertikalen Abstieg. Die vierte Komponente legt den Sollgierwinkel fest: $\psi_{\text{soll}} = a_\psi \cdot 180^\circ$. Die Sollwerte werden nicht direkt an die Motoren übergeben, sondern von einem kaskadierenden PID-Regler in Motordrehzahlen umgesetzt (vgl. Abschnitt 4.6).

4.2.3 Übergangsmodell

Die Übergangsdynamik p wird deterministisch durch den Physiksimulator bestimmt (Schrittweite $\Delta t = 0,01 \text{ s}$, 100 Hz). Der RL-Agent trifft alle 300 Schritte eine Entscheidung (Entscheidungsintervall 3,0 s); dazwischen steuert der PID-Regler die Drohne zum Sollzustand.

4.2.4 Belohnungsfunktion

Die Belohnungsfunktion kombiniert dichte Signale, die den Fortschritt zur Zielerreichung messen, mit spärlichen terminalen Signalen, die das eigentliche Ziel kodieren. Die Fortschrittskomponente ist dabei als potentialbasiertes *Reward Shaping* im Sinne von Ng u. a. [45] konstruiert ($\Phi(s) = -d(s)$); die übrigen dichten Komponenten weichen bewusst davon ab, da in Vorversuchen eine rein potentialbasierte Belohnungsfunktion zu langsamerer Konvergenz führte. Die Belohnungsfunktion setzt sich als gewichtete Summe aus fünf Komponenten zusammen¹:

$$r_t = w_{\text{prog}} r_{\text{prog}} + w_{\text{nah}} r_{\text{nah}} + w_{\text{prox}} r_{\text{prox}} + w_{\text{crash}} r_{\text{crash}} + w_{\text{erf}} r_{\text{erf}} \quad (4.6)$$

Fortschritt. $r_{\text{prog}} = d_t - d_{t+1}$ misst die Annäherung an das Ziel pro Zeitschritt.

Nähe. $r_{\text{nah}} = \exp(-d_{t+1})$ liefert ein exponentiell ansteigendes Signal in Zielnähe und belohnt das Verbleiben nahe dem Ziel.

Dabei bezeichnet $d_t = \|\mathbf{p}_{\text{Ziel}} - \mathbf{p}_t\|$ den euklidischen Abstand zum Ziel.

¹Das Symbol r_t bezeichnet hier die skalare Belohnung; das in Abschnitt 2.2.2 eingeführte Wahrscheinlichkeitsverhältnis wird durch das Argument θ als $r_t(\theta)$ unterschieden.

Hindernisproximität.

$$r_{\text{prox}} = \max\left(1 - \frac{\delta_{\text{min}}}{\delta_s}, 0\right), \quad \delta_s = 3,0 \text{ m} \quad (4.7)$$

bestraft das Eindringen in einen Sicherheitsradius um Hindernisse, wobei δ_{min} den minimalen AABB-Abstand bezeichnet.

Absturz. $r_{\text{crash}} = 1$ bei Verletzung einer Abbruchbedingung (vgl. Abschnitt 4.2.5).

Erfolg. $r_{\text{erf}} = 1$ bei Erreichen des Landeziels.

Die Gewichte (Tabelle 4.2) wurden experimentell abgestimmt. Eine zu hohe Gewichtung der Strafterme führte in Vorversuchen zu stagnierendem Schwebeverhalten. Eine Timeout-Strafe wurde bewusst ausgelassen, da der Agent ohne Zeitbeobachtung im Zustandsvektor den Episodenabbruch nicht antizipieren kann [43].

Tabelle 4.2.: Gewichte der Belohnungskomponenten. Negative Gewichte kennzeichnen Strafterme.

Komponente	Symbol	Gewicht
Fortschritt	w_{prog}	1,0
Nähe	w_{nah}	2,0
Hindernisproximität	w_{prox}	-0,2
Absturz	w_{crash}	-10,0
Erfolg	w_{erf}	20,0

4.2.5 Terminierung

Eine Episode endet bei **Erfolg** (Drohne verbleibt 3,0 s innerhalb von 0,5 m um das Landeziel), **Absturz** (Flughöhe $z < 0,2$ m, Neigung $> 60^\circ$, AABB-Abstand $< 0,3$ m oder Verlassen des Korridors) oder **Timeout** ($T_{\text{max}} = 20$ Entscheidungsschritte, 60 s). Timeouts werden im Vorteilsschätzer als nicht-terminale Übergänge behandelt.

4.3 Netzwerkarchitektur

Es wird eine *Actor-Critic*-Architektur verwendet (vgl. Abschnitt 2.2.1). Der CNN-Encoder besteht aus drei Faltungsschichten mit aufsteigender Kanalzahl (32, 64, 128), abnehmenden Kernelgrößen (8×8 , 4×4 , 3×3) und Schrittweiten (4, 2, 1), jeweils gefolgt von Batch-Normalisierung und Exponential Linear Unit (ELU) Aktivierungsfunktion. Ein abschließendes *Global Max Pooling* erzeugt einen 128-dimensionalen Merkmalsvektor. Der Zustandsvektor durchläuft eine laufende empirische Normalisierung. Der resultierende Vektor wird an die

MLP-Köpfe weitergegeben: Actor mit drei Schichten à 64 Neuronen, Critic mit zwei Schichten à 32 Neuronen (beide ELU). Actor und Critic teilen sich den CNN-Encoder. Der Actor gibt pro Aktionsdimension einen Erwartungswert μ und eine Log-Standardabweichung $\log \sigma$ aus. Aus diesen wird eine Aktion gemäß der Tanh-Gaußverteilung gezogen (Abschnitt 4.4.2). Der Critic gibt einen skalaren Zustandswert aus. Abbildung 4.2 zeigt die vollständige Architektur.

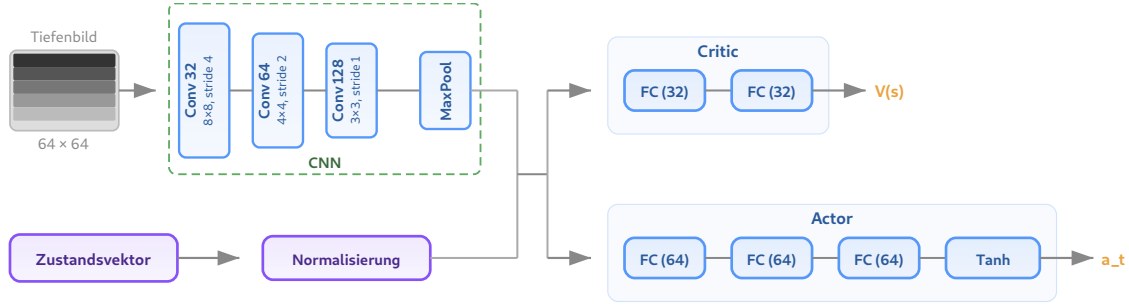


Abbildung 4.2.: Netzwerkarchitektur: Geteilter CNN-Encoder (drei Faltungsschichten mit Batch-Normalisierung, ELU und *Global Max Pooling*) erzeugt einen 128-dimensionalen Merkmalsvektor, der mit dem normalisierten Zustandsvektor konkateniert und an die getrennten MLP-Köpfe weitergegeben wird.

4.4 Algorithmus

4.4.1 Proximal Policy Optimization

Als Trainingsalgorithmus wird PPO [25] eingesetzt (vgl. Abschnitte 2.2.2 und 3.3). Neuere Varianten wie *Phasic Policy Gradient* [46] wurden nicht evaluiert, da die verwendete RL-Bibliothek rsl-rl [47] ausschließlich Standard-PPO bereitstellt.

4.4.2 Aktionsverteilung und Tanh-Squashing

Der Aktionsraum ist auf $[-1, 1]^4$ beschränkt (vgl. Abschnitt 4.2.2). Ein naives Clipping $a_i = \text{clip}(u_i, -1, 1)$ würde die Aktionswerte korrekt begrenzen, die zugehörige Log-Wahrscheinlichkeit jedoch weiterhin auf dem unbeschnittenen Wert u_i berechnen. Diese Diskrepanz zwischen ausgeführter und bewerteter Aktion führt zu verzerrten Gradienten.

Statt Clipping wird eine invertierbare Transformation [48] verwendet:

$$a_i = \tanh(u_i), \quad u_i \sim \mathcal{N}(\mu_i, \sigma_i^2). \quad (4.8)$$

Die zugehörige Log-Wahrscheinlichkeit ergibt sich über die Substitutionsregel:

$$\log \pi(\mathbf{a} \mid s) = \log \mathcal{N}(\mathbf{u} \mid \boldsymbol{\mu}, \boldsymbol{\sigma}^2) - \sum_{i=1}^4 \log(1 - \tanh^2(u_i)). \quad (4.9)$$

Zusätzlich erzwingt ein Floor $\log \sigma_i \geq -2,0$ ($\sigma_i \geq 0,135$) eine Mindestexploration.

4.4.3 Hyperparameter

Tabelle 4.3 fasst die PPO-Hyperparameter zusammen. Die Werte für ε , γ , λ und K entsprechen den Standardwerten [25, 27].

Tabelle 4.3.: Hyperparameter des PPO-Algorithmus.

Parameter	Wert
Clipping ε	0,2
Diskontierung γ	0,99
GAE λ	0,95
Lern-Epochen K	5
Mini-Batches M	4
c_1 (Value-Loss)	1,0
c_2 (Entropie)	0,003
Gradient-Clipping	1,0

4.5 Trainingsumgebung

4.5.1 Physiksimulator

Die vorliegende Arbeit verwendet Genesis [49], einen quelloffenen, GPU-parallelierten Physiksimulator für Robotik. Genesis führt Simulation tausender Umgebungen GPU-parallel aus, stellt Tiefenkameras bereit und importiert beliebige 3D-Meshes. Gegenüber anderen gängigen Simulatoren wie Gazebo [50] und PyBullet [51] bietet Genesis GPU-Parallelisierung. Gegenüber dem proprietären Isaac Gym [52] bietet Genesis eine geringere Einstiegskomplexität.

4.5.2 Drohnenmodell

Die Drohne wird als URDF-Modell (Unified Robot Description Format) mit vier Rotoren simuliert (Masse 0,714 kg, Schub-Gewicht-Verhältnis 2,25, Hover-Drehzahl 1789 RPM, maximale Drehzahl 2700 RPM). Abbildung 4.3 zeigt das simulierte Drohnenmodell.

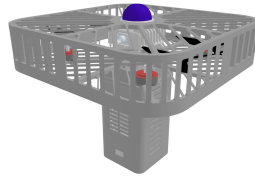


Abbildung 4.3.: Simuliertes Drohnenmodell mit Schutzkäfig und vier Rotoren.

4.5.3 Korridorgeometrie

Die Trainingsumgebung bildet einen vertikalen Korridor der Ausdehnung $5 \times 2 \times 15$ m ($x \times y \times z$) ab (Abbildung 4.4). Der Korridor besitzt keine physischen Wände; das Verlassen des Begrenzungsrahmens wird als Absturz gewertet. Die Drohne wird bei $z = 12$ m mit zufälliger horizontaler Position innerhalb von $\pm 1,5$ m (x) bzw. $\pm 0,5$ m (y) initialisiert. Das Landeziel ist fixiert bei $\mathbf{p}_{\text{Ziel}} = (0, 0, 1)^T$ m. Während des Abstiegs durchquert die Drohne zwei horizontale Hindernisschichten bei $z \approx 2$ m und $z \approx 7$ m.

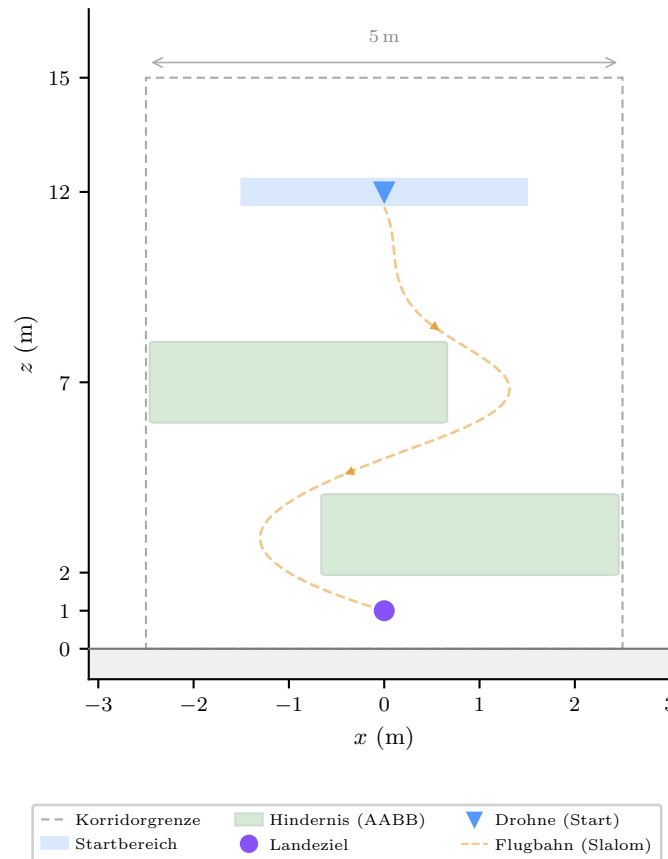


Abbildung 4.4.: Schematische Seitenansicht der Korridorgeometrie (xz -Ebene) mit Slalom-Flugbahn.

4.5.4 Hindernisse

Als Hindernisse dienen sechs 3D-Objekte aus dem *Google Scanned Objects* Datensatz (GSO) [53], jeweils um den Faktor 13 bis 63 skaliert, sodass ihre horizontale Ausdehnung die gesamte

y-Breite des Korridors abdeckt. Für die Kollisionserkennung und die Hindernisproximität (vgl. Abschnitt 4.2.4) wird die achsenparallele Begrenzungsbox (AABB) jedes Objekts herangezogen. Diese konservative Approximation umschließt das Mesh vollständig, sodass die Drohne bereits vor Berührung der tatsächlichen Oberfläche als kollidiert gilt. Eine oberflächenbasierte Kontakterkennung wäre geometrisch präziser, erfordert jedoch pro Zeitschritt Kollisionsabfragen gegen die vollständigen Mesh-Geometrien aller Hindernisse, was bei 512 parallelen Umgebungen den Trainingsdurchsatz erheblich reduziert. Die Platzierung folgt einem erzwungenen Slalom entlang der x -Achse: Pro Hindernisschicht wird ein zufällig gewähltes Objekt auf einer Seite der Korridormitte positioniert (0,6 bis 1,2 m Versatz); aufeinanderfolgende Schichten wechseln die Seite. Abbildung 4.5 zeigt eine Simulationsansicht während eines Abstiegs.

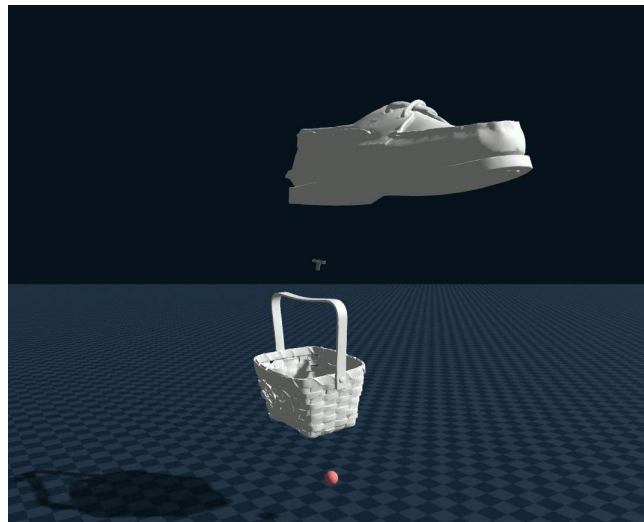


Abbildung 4.5.: Simulationsansicht mit GSO-Hindernissen während des Trainings. Die Drohne navigiert zwischen zwei skalierten Mesh-Objekten zum Landeziel (rot).

4.6 Kaskadierender PID-Regler

Der RL-Agent gibt Sollposition und Sollgierwinkel vor. Ein dreistufiger kaskadierender PID-Regler (Proportional-Integral-Derivative; Position $\rightarrow$ Geschwindigkeit $\rightarrow$ Lage) setzt diese in Motordrehzahlen um. Die Verstärkungen sind in Tabelle I1 im Anhang aufgeführt. Der Regler wurde eigens für diese Simulationsumgebung entwickelt, da ein etablierter Flugreglerstack wie PX4 [33] im verwendeten Simulator nicht zur Verfügung stand.

4.7 Trainingsprozess

Tabelle 4.4 fasst die Trainingsparameter zusammen. Das Training wird auf dem HPC-Cluster (High-Performance Computing) der Universität Leipzig mit einer NVIDIA A30 (Ampere-Architektur, 24 GB VRAM) durchgeführt. Zur Überprüfung der Reproduzierbar-

keit werden sechs Trainingsläufe mit verschiedenen Zufallsinitialisierungen durchgeführt. Die Checkpoint-Selektion erfolgt anhand des Belohnungsverlaufs (*Early Stopping*).

Tabelle 4.4.: Konfiguration des Trainingsprozesses.

Parameter	Wert
Parallele Umgebungen N	512
Schritte pro Umgebung T	60
Batch-Größe $N \times T$	30 720
Lernrate α	$1 \times 10^{-3} \rightarrow 0$ (linearer Abfall)
Optimizer	Adam
Trainingsiterationen	1 000
GPU	NVIDIA A30 (24 GB)
Laufzeit	~24 h
Checkpoint-Intervall	100 Iterationen

4.8 Evaluation

Die Evaluation untersucht die trainierte Strategie anhand zweier Versuchsreihen mit jeweils 1 000 Episoden und vergleicht sie mit einem RRT-Connect-Pfadplaner (*Rapidly-exploring Random Tree*).

4.8.1 Evaluationsmetriken

Alle Metriken werden mit 95 %-Bootstrap-Konfidenzintervallen ($B = 10\,000$, Perzentilmethode) [54] versehen. Erhoben werden: **Erfolgsrate**, **Fehlerartenverteilung** (Hinderniskollision, Bodenkollision, Korridorverlassen, Timeout), **Landezeit** (Median mit Interquartilsabstand (IQR), nur erfolgreiche Episoden), **minimaler Hindernisabstand** (AABB-basiert, Mittelwert), **Pfادهffizienz**

$$\eta_{\text{Pfad}} = \frac{\sum_{t=0}^{T-1} \|\mathbf{p}_{t+1} - \mathbf{p}_t\|}{\|\mathbf{p}_{\text{Ziel}} - \mathbf{p}_0\|} \quad (4.10)$$

(Median mit IQR, nur erfolgreiche Episoden).

4.8.2 Versuchsreihe 1: Generalisierung auf unbekannte Hindernisformen

Die erste Versuchsreihe prüft die Generalisierung auf zuvor ungesehene Hindernisformen: Die sechs Trainingsobjekte werden durch sechs neue GSO-Objekte gleicher Skalierung ersetzt; alle übrigen Parameter bleiben identisch. Als Vergleichsverfahren dient ein RRT-Connect-Pfadplaner mit vollständigem Umgebungswissen (wahre Hindernispositionen und -geometrien). Die Leistungsdifferenz quantifiziert den Preis eingeschränkter Sensorik, da

der RL-Agent ausschließlich ein lokales Tiefenbild und den propriozeptiven Zustandsvektor erhält.

4.8.3 Versuchsreihe 2: Transfer in eine realistische Agrarumgebung

Die zweite Versuchsreihe untersucht den *Zero-Shot*-Transfer in eine photogrammetrisch erfasste Weinbergumgebung (Eltville, Deutschland, vierfache Skalierung, Abbildung 4.6). Die Kollisionserkennung erfolgt über ein vorberechnetes Höhenraster mit einem Sicherheitsabstand von 0,5 m; die AABB-basierte Hinderniskollision und die Korridorgrenzenprüfung entfallen. Als Landeziele dienen 20 vordefinierte Positionen (15 zwischen Reihen, 5 auf offenem Gelände). Die Drohne wird in variabler Höhe (10 bis 15 m) über dem Ziel initialisiert. Abbildung 4.7 zeigt die Zielpositionen.

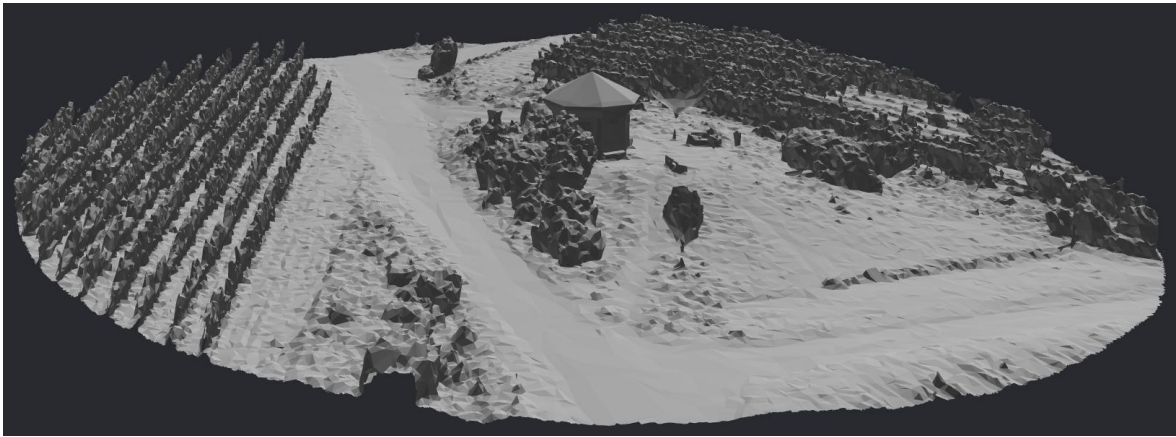


Abbildung 4.6.: Photogrammetrisch erfasstes Weinberg-Mesh (Eltville, Deutschland) in vierfacher Skalierung. Die diagonalen Strukturen sind die Reihen, zwischen denen die Landeziele platziert sind.

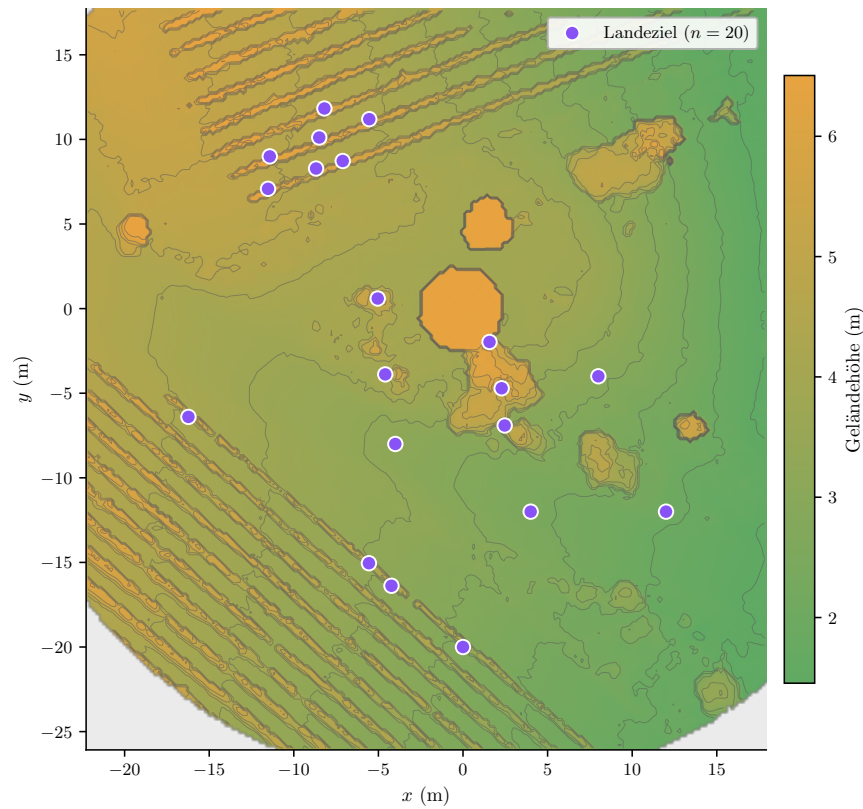


Abbildung 4.7.: Draufsicht auf das Höhenraster der Weinbergumgebung mit den 20 vordefinierten Zielpositionen. Die diagonalen Strukturen entsprechen den Rebenreihen.

5 Ergebnisse

5.1 Trainingsverhalten

Das Training erreicht einen Durchsatz von 341 FPS. Jeder der sechs Läufe (vgl. Abschnitt 4.7) wurde nach 24 Stunden durch die GPU-Zeitbegrenzung beendet. Abbildung 5.1 zeigt die Belohnungsverläufe aller sechs Seeds.

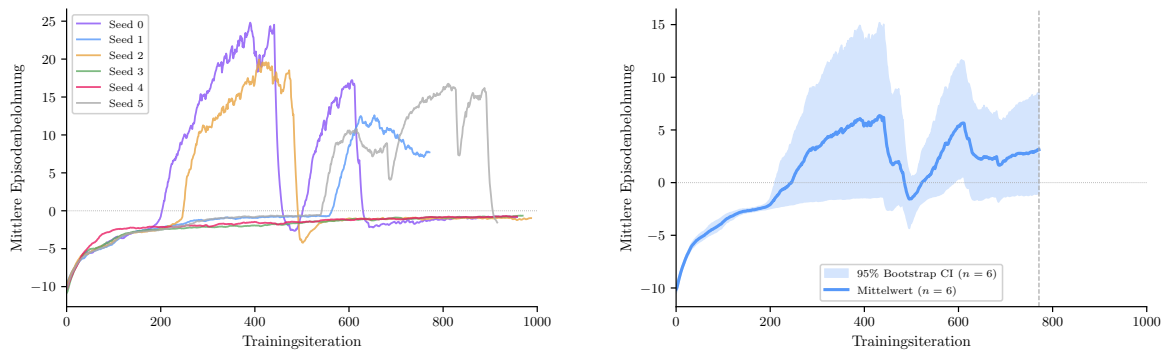


Abbildung 5.1.: Mittlere Episodenbelohnung über 6 Seeds mit identischen Hyperparametern.
Links: Einzelne Seeds mit vollständigem Trainingsverlauf bis zum jeweiligen Abbruch. Rechts: Mittelwert mit 95 %-Bootstrap-Konfidenzintervall (10 000 Resamples); die gestrichelte Linie markiert das Ende des kürzesten Laufs. EMA $\alpha = 0,9$.

Von den sechs Seeds konvergieren vier zu einer funktionsfähigen Strategie (Tabelle 5.1). Zwei Seeds (3 und 4) erzielen in über 900 Iterationen keine einzige erfolgreiche Landung, obwohl sie Absturzvermeidung erlernen (sinkende Absturzrate). Die konvergierenden Seeds zeigen einen charakteristischen Verlauf: Die kumulative Belohnung steigt zunächst durch die dichten Belohnungskomponenten (Anhang A zeigt die Einzelverläufe für Seed 0). Ab der ersten erfolgreichen Landung (zwischen Iteration 202 und 594, je nach Seed) beschleunigt die Erfolgsbelohnung den Anstieg. Die Standardabweichung der Aktionsverteilung fällt bei allen sechs Seeds nahezu identisch von 0,99 auf circa 0,15, unabhängig vom Belohnungsverlauf (Abbildung B1 im Anhang).

Tabelle 5.1.: Konvergenzergbnis der 6 Seeds. Höchstwert: höchste EMA-geglättete Belohnung ($\alpha = 0,95$). Kollaps: Iteration des abrupten Belohnungseinbruchs (> 10 Punkte innerhalb von 50 Iterationen).

Seed	Höchstwert	Iter.	Konvergenz	Kollaps
0 (Final)	23,5	391	ja	Iter. 445
1	11,8	654	ja	–
2	19,0	429	ja	Iter. 484
3	–0,7	–	nein	–
4	–0,7	–	nein	–
5	16,2	811	ja	Iter. 897

Drei der vier konvergierenden Seeds zeigen einen abrupten Belohnungseinbruch, jeweils 50–85 Iterationen nach dem Höchstwert. Bei Seed 0 fällt die Belohnung innerhalb einer Iteration von 29,2 auf –6,1. Die Standardabweichung bleibt dabei nahezu unverändert (0,193 $\rightarrow$ 0,194). Der Kollaps betrifft die Mittelwerte der Strategie, nicht die Explorationsbreite.

Der beste Checkpoint liegt bei Iteration 400 von Seed 0 (10,4 Stunden Trainingszeit, kumulative Belohnung 23,0; Tabelle 5.2). Seed 0 stellt den leistungsstärksten der sechs Seeds dar. Die Evaluationsergebnisse in den folgenden Abschnitten bilden daher eine günstige, nicht die erwartete Leistung ab. Auf Basis des Belohnungsverlaufs wurde dieser Checkpoint für alle nachfolgenden Evaluierungen ausgewählt (*Early Stopping*).

Tabelle 5.2.: Checkpoint-Selektion für Seed 0 (512 Umgebungen, NVIDIA A30, 341 FPS).

Checkpoint	Iteration	Laufzeit	Kum. Belohnung
Bestes Modell	400	10,4 h	23,0
Post-Kollaps	900	22,6 h	–1,1
Trainingsende	957	24,0 h	–0,8

5.2 Phase 1: Korridornavigation

Versuchsreihe 1 (vgl. Abschnitt 4.8.2) evaluiert die trainierte Strategie (Checkpoint 400) über 1 000 Episoden mit zuvor ungesehenen Hindernisobjekten. Der Agent erreicht eine Erfolgsrate von 71,0 % (Abbildung 5.2). Die fehlgeschlagenen Episoden verteilen sich nahezu vollständig auf Hinderniskollisionen (28,8 %); die verbleibenden 0,2 % enden durch Verlassen des Korridors. Bodenkollisionen und Timeouts treten nicht auf. Tabelle C1 im Anhang listet alle Metriken mit Konfidenzintervallen.

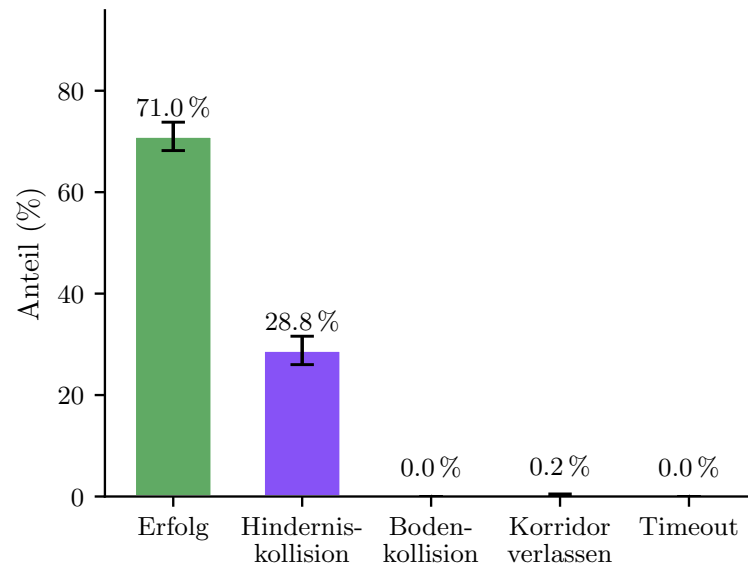


Abbildung 5.2.: Episodenergebnisse der Phase-1-Evaluation (1 000 Episoden). Fehlerbalken zeigen 95 %-Bootstrap-Konfidenzintervalle.

Das Flugverhalten zeigt zwei wiederkehrende Muster (Abbildung 5.3): Bei freier Bahn vollzieht der Agent einen annähernd vertikalen Abstieg (mediane Pfadeffizienz 1,19). In der Nähe eines Hindernisses bremst er spät ab und weicht lateral mit geringem Sicherheitsabstand aus (mittlerer Minimalabstand 0,52 m).

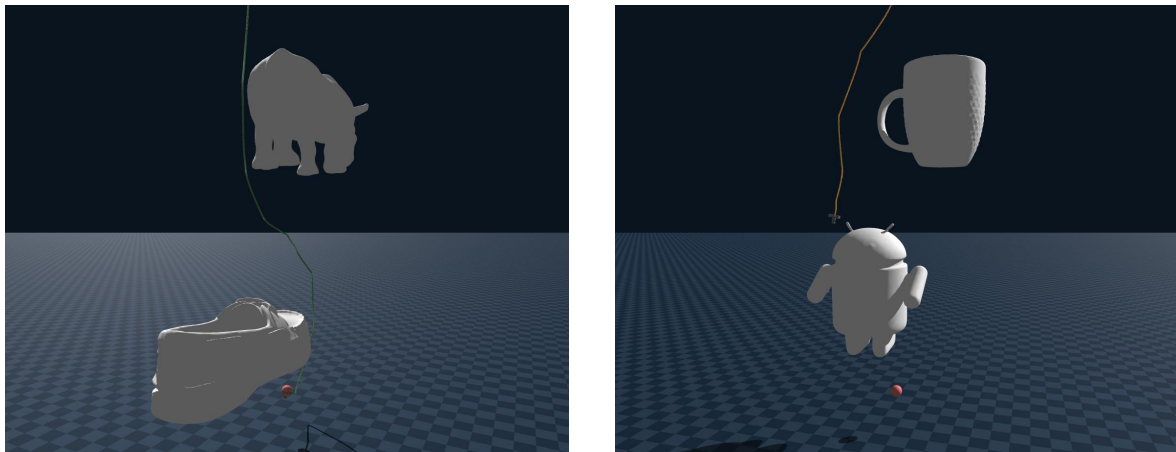


Abbildung 5.3.: Simulationsansicht einer erfolgreichen Landung (links) und einer Hinderniskollision (rechts). Die Flugbahn ist als farbige Linie dargestellt; GSO-Hindernisse sind in ihrer tatsächlichen Geometrie sichtbar.

5.2.1 Nachuntersuchung: Oberflächenbasierte Kollisionserkennung

Das knappe Passieren bei hoher Kollisionsrate legte nahe, dass die konservative AABB-Erkennung einen Teil der Fehlschläge verursacht. In einer Nachuntersuchung mit 1 000 Episoden wurde die AABB-Kollisionserkennung durch die geometrisch exakte Oberflächenkollisionsprüfung des Simulators ersetzt.

Mit oberflächenbasierter Kontakterkennung steigt die Erfolgsrate auf 98,4 % (Tabelle 5.3). Ein erheblicher Anteil der AABB-Kollisionen sind demnach Fehlalarme: Die Drohne durchfliegt die Begrenzungsbox, ohne die Oberfläche zu berühren. Tabelle D1 im Anhang listet alle Metriken.

Tabelle 5.3.: Vergleich AABB-Distanz und oberflächenbasierte Kontakterkennung ($n = 1\,000$).

Metrik	AABB	Oberfläche
Erfolgsrate	71,0 %	98,4 %
Hinderniskollision	28,8 %	1,2 %
Verlassen des Korridors	0,2 %	0,2 %
Timeout	0,0 %	0,2 %

5.3 Phase 2: Weinberg-Transfer

Versuchsreihe 2 (vgl. Abschnitt 4.8.3) evaluiert den *Zero-Shot*-Transfer in die Weinbergumgebung über 1 000 Episoden. Der Agent erreicht eine Erfolgsrate von 10,4 % (Abbildung 5.4). Timeouts dominieren mit 73,7 %; Terrain-Kollisionen verursachen 15,9 %. Der mediane Abstand zum Landeziel am Episodenende beträgt 1,19 m; 59,7 % der Timeout-Episoden befinden sich innerhalb von 1,5 m zum Ziel. Tabelle F1 im Anhang listet alle Metriken.

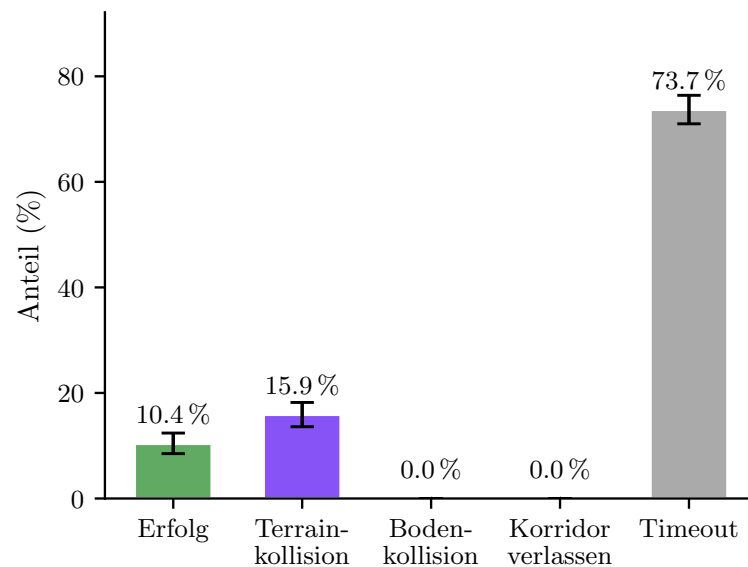


Abbildung 5.4.: Episodenergebnisse der Phase-2-Evaluation (1 000 Episoden, *Zero-Shot*-Transfer). Fehlerbalken zeigen 95 %-Bootstrap-Konfidenzintervalle.

In der Mehrzahl der Fälle erreicht der Agent die Umgebung des Landeziels, verbleibt jedoch außerhalb des Erfolgsradius von 0,5 m mit ineffektiven Korrekturbewegungen.

5.3.1 Sensitivitätsanalyse des Erfolgskriteriums

Mit einem erweiterten Erfolgswert von 1,5 m steigt die Erfolgsrate auf 69,7 % (Tabelle 5.4). Die Terrain-Kollisionsrate sinkt von 15,9 % auf 9,4 %, da ein Teil der zuvor als Timeout gewerteten Episoden nun frühzeitig als Erfolg beendet wird. Der Timeout-Anteil sinkt von 73,7 % auf 27,7 %.

Tabelle 5.4.: Sensitivitätsanalyse des Erfolgswertes r in der Weinbergumgebung ($n = 1\,000$ je Bedingung).

Metrik	$r = 0,5\text{ m}$	$r = 1,5\text{ m}$
Erfolgsrate	10,4 %	69,7 %
Terrain-Kollision	15,9 %	9,4 %
Timeout	73,7 %	27,7 %

5.4 Vergleich RL vs. RRT-Connect

Der RRT-Planer übertrifft den RL-Agenten in der Erfolgsrate beider Umgebungen, wobei der Abstand im Weinberg deutlich größer ausfällt. Im Korridor dominieren bei beiden Verfahren Hinderniskollisionen; im Weinberg scheitert der RL-Agent überwiegend an Timeouts (73,7 %), während der RRT-Planer diese auf 7,9 % begrenzt. Der RL-Agent landet schneller (30,0 s gegenüber 42,0 s im Korridor).² Dem steht ein einmaliger Trainingsaufwand von circa 24 Stunden gegenüber. Detaillierte Metriken mit Konfidenzintervallen finden sich in den Anhangstabellen C1 und F1 (RL) sowie E1 und G1 (RRT-Connect). Tabelle 5.5 stellt die Ergebnisse beider Verfahren gegenüber.

Tabelle 5.5.: Vergleich von RL-Agent und RRT-Connect über beide Versuchsreihen ($n = 1\,000$ je Bedingung).

Metrik	Phase 1 (Korridor)		Phase 2 (Weinberg)	
	RL	RRT	RL	RRT
Erfolgsrate	71,0 %	82,0 %	10,4 %	85,7 %
(relaxed $r = 1,5\text{ m}$)	n. a.		69,7 %	98,4 %
Hindernis-/Terrain-Koll.	28,8 %	18,0 %	15,9 %	7,1 %
Timeout	0,0 %	0,0 %	73,7 %	7,9 %
Landezeit (Median)	30,0 s	42,0 s	36,0 s	42,0 s
Pfادهffizienz (Median)	1,19	1,09	1,07	0,99
Min. Abstand (Mittel)	0,52 m	0,48 m	1,02 m	0,46 m

²Einzelne Pfادهffizienzwerte knapp unter 1,0 sind ein Implementierungsartefakt: Das letzte Flugsegment wird aufgrund des automatischen Zurücksetzens der Umgebung nicht erfasst.

6 Diskussion

6.1 Leistungsfähigkeit in der Trainingsumgebung

6.1.1 Trainingsverhalten und Konvergenz

Die zwei Phasen im Belohnungsverlauf lassen sich auf die Belohnungsstruktur zurückführen. In der Konvergenzphase (Iteration 0 bis 441) ermöglichen die dichten Komponenten (r_{prog} , r_{nah} , r_{prox}) den initialen Anstieg; ab circa Iteration 200 setzt die Erfolgsbelohnung ein und beschleunigt den Anstieg. Ohne die dichten Komponenten würde dem Agenten das Gradientensignal für Teilfortschritte fehlen.

Der abrupte Kollaps tritt bei 3 von 6 Seeds auf (Iterationen 445, 484, 897), jeweils 50–85 Iterationen nach dem Belohnungsmaximum, bei nahezu unveränderter Standardabweichung. Die Reproduzierbarkeit des Musters spricht für den von Moalla u. a. [55] beschriebenen Mechanismus: Eine schleichende Verschlechterung des Feature-Ranks im Actor-Netzwerk untergräbt den Clipping-Mechanismus von PPO [25], bis eine einzelne Aktualisierung die Strategie irreversibel destabilisiert. Dass die Standardabweichung als globaler Parameter unbeeinflusst bleibt, ist mit diesem Mechanismus konsistent. Eine schrittweise Reduktion der Lernrate verhinderte den Kollaps nicht. Moalla u. a. [55] schlagen mit *Proximal Feature Optimization* eine Regularisierung vor, die die Feature-Diversität im Netzwerk aktiv erhält und den Rang-Verfall adressiert.

Die zwei nicht-konvergierenden Seeds zeigen eine andere Problematik: Sie lernen Absturzvermeidung, erreichen jedoch nie eine erfolgreiche Landung. Die Erfolgsbelohnung (+20) wirkt als Phasenübergang: Seeds, die bis circa Iteration 300 keine Landung erzielen, erhalten das selbstverstärkende Gradientensignal nie. Nikishin u. a. [56] beschreiben einen verwandten Effekt, bei dem frühe Erfahrungssequenzen die Repräsentation überproportional formen und späteres Lernen blockieren. Als Gegenmaßnahme schlagen sie periodische Reinitialisierungen der letzten Netzwerkschichten vor, die den festgefahrenen Repräsentationen eine erneute Anpassung ermöglichen.

Die im Trainingsverlauf ansteigende Landezeit lässt sich auf die Belohnungsstruktur zurückführen. $r_{\text{prog}} = d_t - d_{t+1}$ ist potentialbasiert im Sinne von Ng u. a. [45] und verändert die optimale Strategie nicht. $r_{\text{nah}} = \exp(-d_{t+1})$ hingegen ist nicht potentialbasiert: Jeder Zeitschritt in Zielnähe liefert Belohnung, unabhängig von weiterer Annäherung. Da es keine Zeitstrafe gibt, überwiegt der akkumulierte Schwebe-Anreiz. Dieses Phänomen dokumen-

tieren auch Randløv und Alstrøm [57]: dass eine nicht-potentialbasierte Zwischenbelohnung einen Agenten dazu bringt, Kreise zu fahren statt das Ziel zu erreichen, da die akkumulierte Zwischenbelohnung die Zielbelohnung übersteigt. Entsprechend lernt der Agent eine Strategie, die steilen Abstieg mit ausgedehntem Schweben in Zielnähe kombiniert, was die beobachtete Zunahme der Episodenlänge im Trainingsverlauf erklärt. Eine Umformulierung von r_{nah} als potentialbasierte Belohnung würde den Schweben-Anreiz beseitigen, ohne die Lernbeschleunigung aufzugeben.

6.1.2 Dominante Fehlerart und Flugverhalten

Hinderniskollisionen (28,8 %) sind die einzig relevante Fehlerart. Mit oberflächenbasierter Kollisionserkennung (Abschnitt 5.2.1) steigt die Erfolgsrate auf 98,4 %. Die Diskrepanz erklärt sich aus drei Faktoren: Erstens ist die Hindernisproximität ($w_{\text{prox}} = -0,2$) deutlich schwächer gewichtet als Fortschritt und Nähe, sodass der Agent knappes Passieren rational in Kauf nimmt. Zweitens beschränkt das Entscheidungsintervall von 3 s die Reaktionszeit. Drittens erzwingt die nach unten gerichtete Kamera ein reaktives Flugverhalten, bei dem Hindernisse erst bei Annäherung erkannt werden. Das Flugmuster (direkter Abstieg, spätes Abbremsen, laterales Ausweichen mit mittlerem Minimalabstand von 0,52 m) ist präzise genug, um die tatsächliche Oberfläche zu verfehlen, durchdringt jedoch regelmäßig die konservative AABB-Hülle. Das Flugverhalten legt eine funktionale Rollenverteilung nahe: Die lateralen Ausweichmanöver setzen Hindernisinformation aus dem Tiefenbild voraus. Der Zustandsvektor steuert Zielnavigation und Abstieg.

Zur Einordnung: Bartolomei u. a. [15] berichten vergleichbare Raten (über 70 %) für ein RL-System mit Tiefenbildeingabe, allerdings für die Selektion sicherer Landezonen, nicht für die Navigation durch enge Hindernispassagen.

6.2 Transferleistung auf den Weinberg

6.2.1 Leistungsabfall und Verschiebung der Fehlerarten

Ein Leistungsabfall beim Umgebungswechsel ist erwartbar, da der Agent die Weinberggeometrie erstmals zur Evaluationszeit sieht. Aufschlussreich ist die Fehlerverschiebung: Während in Phase 1 Hinderniskollisionen dominieren, verschiebt sich die Verteilung in Phase 2 zu Zeitüberschreitungen (73,7 %) bei geringer Terrain-Kollisionsrate (15,9 %). Der Agent erreicht die Zielumgebung (mediane finale Distanz 1,19 m), kann dort jedoch nicht konvergieren. Mit erweitertem Erfolgsradius von 1,5 m steigt die Erfolgsrate auf 69,7 %.

Die Ursache liegt in der strukturell veränderten Landesituation: Im Korridor ist die Landezone hindernisfrei; im Weinberg ragen Reihen direkt in die Landeumgebung. Dass die Aufgabe grundsätzlich lösbar ist, zeigt der RRT-Connect-Planer mit 85,7 % (vgl. Abschnitt 6.3).

6.2.2 Einordnung als Sim-to-Sim Transfer

Dass die Navigationsleistung dennoch transferiert (69,7 % bei erweitertem Radius), lässt sich auf die umgebungsunabhängige Beobachtungsstruktur zurückführen: Der Zustandsvektor kodiert relative Positionen, und das Tiefenbild als abstrakte Zwischenrepräsentation den Domänenunterschied zwischen Umgebungen reduziert [15, 16]. *Sim-to-Real*-Arbeiten berichten vergleichbare Einbußen: Polvara u. a. [18] verlieren 28 Prozentpunkte, Ladosz u. a. [19] 27 Prozentpunkte. Der hier beobachtete nominale Abfall von 61 Prozentpunkten relativiert sich bei erweitertem Radius auf 1,3 Prozentpunkte, was zeigt, dass der Umgebungswechsel die terminale Landephase betrifft, nicht die Navigation.

6.2.3 Perceptual Aliasing als Erklärung für die Terrain-Kollisionen

Ein möglicher Mechanismus hinter den 15,9 % Terrain-Kollisionen ist *Perceptual Aliasing* [58]: Die Kamera erzeugt für Boden und Vegetation bei Annäherung ähnliche Signaturen (Abbildung 6.1). In der Trainingsumgebung ist diese Mehrdeutigkeit unproblematisch, da die Landezone hindernisfrei ist. Im Weinberg bricht diese Strategie: Reihen ragen in die Landezone, sodass ein Agent, der das Tiefenbild in Zielnähe zu ignorieren gelernt hat, mit Vegetation kollidiert. Bartolomei u. a. [15] adressieren dieses Problem durch zusätzliche semantische Segmentierungen als Eingabe, welche es dem Agenten erlauben, Hindernisse und Boden zu unterscheiden.

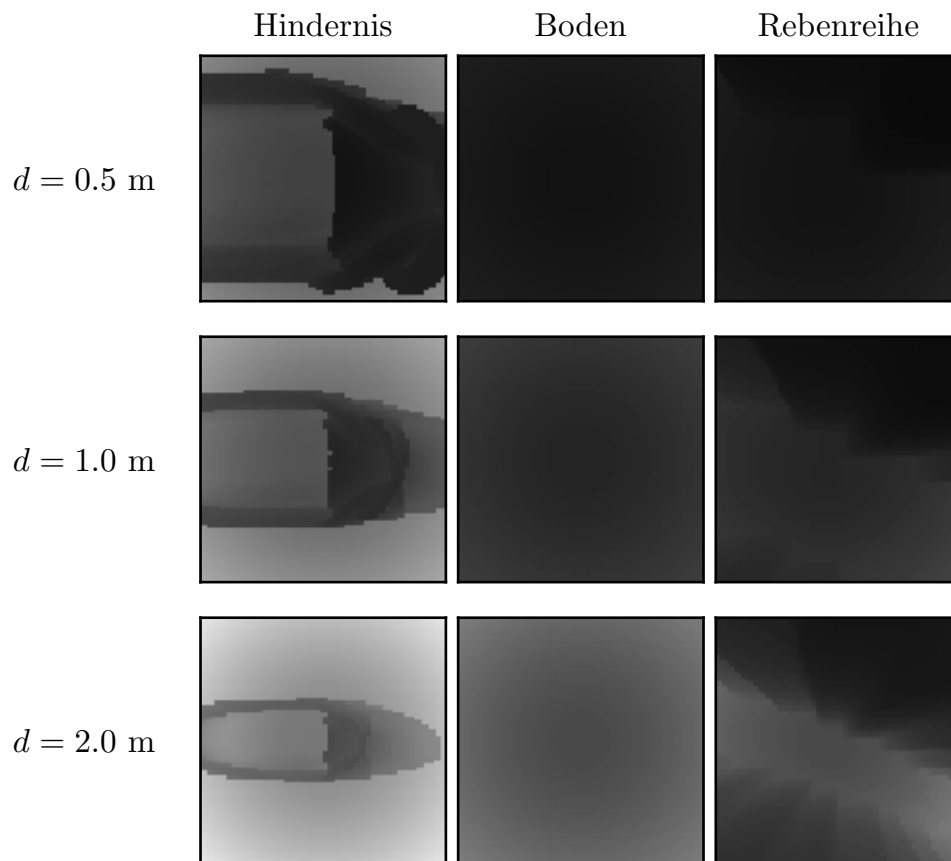


Abbildung 6.1.: Tiefenbilder der nach unten gerichteten Kamera bei drei Distanzen. Boden und Rebenreihe erzeugen deutlich ähnliche Signaturen als Korridorhindernisse.

Verschärft wird das Problem durch die fehlende zeitliche Gedächtnisstruktur: Der Agent verarbeitet nur das aktuelle Tiefenbild. Laterales Ausweichen führt im Weinberg systematisch in die nächste Rebenreihe, da benachbarte Reihen außerhalb des Sichtfelds liegen. Singla u. a. [59] zeigen, dass rekurrente Architekturen mit einem Aufmerksamkeitsmechanismus über vergangene Beobachtungen dieses Problem adressieren können, indem sie vergangene Tiefenbeobachtungen über die Zeit integrieren und so das begrenzte Sichtfeld kompensieren.

6.3 Leistungslücke zum Pfadplaner

Der RRT-Connect-Planer übertrifft den RL-Agenten in der Erfolgsrate: um 11 Prozentpunkte im Korridor und 75 Prozentpunkte im Weinberg. Dieser Vergleich quantifiziert den Preis minimaler Sensorik, nicht die algorithmische Überlegenheit des Planers. Unter oberflächenbasierter Kollisionserkennung erreicht der RL-Agent 98,4 % und liegt damit über der Planerleistung von 82,0 %.

Der RL-Agent landet zudem schneller (30,0 s gegenüber 42,0 s im Korridor), da er direkte Abstiegswege mit knappem Ausweichen lernt, während der Planer konservativere Wege berechnet.

Dass der RL-Agent unter fairer Kollisionsmetrik das Navigationsproblem vergleichbar mit einem vollständig informierten Planer löst, zeigt, dass lernbasierte Ansätze begrenzte Sensorausstattung algorithmisch kompensieren können.

6.4 Limitationen

6.4.1 Trainingssetup

Die Multi-Seed-Analyse (6 Seeds, identische Hyperparameter) zeigt eine Konvergenzrate von 67 % mit hoher Varianz zwischen den Seeds (95 %-Bootstrap-CI-Breite bis 14,8 Punkte bei Iteration 400). Seed 0 liegt als leistungsstärkster Seed im oberen Bereich der Verteilung. Die Evaluationsergebnisse stellen daher eine günstige, nicht die erwartete Leistung dar. Die Belohnungsgewichte wurden manuell in Vorversuchen abgestimmt. Bereits kleine Änderungen können das Flugverhalten qualitativ verändern (vgl. Abschnitt 6.1).

6.4.2 Sensorik und Architektur

Die AABB-Kollisionserkennung überschätzt das Hindernisvolumen systematisch; die Diskrepanz zwischen 71,0 % (AABB) und 98,4 % (oberflächenbasiert) zeigt, dass die berichtete Erfolgsrate die tatsächliche Navigationsleistung unterschätzt. Wie in Abschnitt 4.5 dargestellt, war die oberflächenbasierte Variante im Training aufgrund des Rechenaufwands nicht einsetzbar. Die AABB-Metrik stellt daher einen bewussten Trade-off zwischen Genauigkeit und Trainingseffizienz dar. Der Agent verarbeitet nur das aktuelle Tiefenbild ohne zeitliches Gedächtnis, sodass Strukturen außerhalb des Sichtfelds nicht über mehrere Zeitschritte akkumuliert werden können (vgl. Abschnitt 6.2). Der individuelle Beitrag von Tiefenwahrnehmung und Zustandsvektor wurde nicht durch eine Ablationsstudie quantifiziert. Die Schlussfolgerungen über die gelernten CNN-Repräsentationen sind indirekt aus Trajektorien und Transferleistung abgeleitet.

6.4.3 Umgebungsdiversität

Im Training und in Phase 1 ist das Landeziel in allen Episoden identisch. Erst die Weinberg-Evaluation nutzt 20 verschiedene Zielpositionen. Die Simulation bietet perfekte Sensorik ohne Rauschen, keine Motorverzögerung und keine Kommunikationslatenz. Die Ergebnisse

sind nicht direkt auf reale Drohnen übertragbar. Die Einführung von *Domain Randomization*, etwa durch simuliertes Sensorrauschen und variierte Dynamikparameter, könnte einen künftigen *Sim-to-Real*-Transfer vorbereiten [19, 18].

7 Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

Die vorliegende Arbeit untersuchte, inwiefern ein RL-basierter Ansatz eine autonome Drohne befähigen kann, mithilfe lokaler Tiefensensorik in hindernisreichen Umgebungen sicher zu landen.

Der trainierte Agent erlernt eine reaktive Ausweichstrategie, deren Leistung primär durch strukturelle Entwurfsentscheidungen begrenzt wird: Sensorhorizont, Entscheidungsintervall und Belohnungsgewichtung bestimmen, wie knapp der Agent Hindernisse passiert. Trotz Training unter der konservativen AABB-Kollisionsmetrik erlernt der Agent eine Strategie, die unter oberflächenbasierter Erkennung 98,4 % erreicht und damit den privilegierten Pfadplaner (82,0 %) übertrifft.

Die Transferuntersuchung zeigt eine klare Trennung zweier Teilfähigkeiten: Hindernisnavigation generalisiert auf die Weinbergumgebung (69,7 % bei erweitertem Erfolgswert); die terminale Landephase scheitert an der im Training nie erfahrenen Konfiguration von Vegetation in der Landezone, verstärkt durch *Perceptual Aliasing* und fehlendes zeitliches Gedächtnis.

Der Vergleich mit dem privilegierten Pfadplaner quantifiziert den Vorteil vollständiger Umgebungskennntnis. Unter fairer Kollisionsmetrik löst der lernbasierte Ansatz das Navigationsproblem im Korridor erfolgreicher als der Planer. Im Weinberg kehrt sich dieses Verhältnis um. Die Leistungslücke reflektiert die begrenzte Trainingsdiversität.

Insgesamt zeigt die Arbeit, dass ein RL-Agent mit minimaler Sensorik die kombinierte Aufgabe aus Zielnavigation und Hindernisvermeidung in einer abstrakten Simulationsumgebung lösen kann. Die erlernte Strategie lässt sich teilweise auf ein realistisches Szenario mit natürlicher Vegetation übertragen. Gleichzeitig erfordert das Training eine Checkpoint-Selektion über mehrere Seeds: 4 von 6 Seeds konvergieren, wobei die Mehrzahl einen katastrophalen Leistungseinbruch nach dem Optimum zeigt. Die verbleibenden Leistungsgrenzen liegen weniger im Lernverfahren als in den Randbedingungen des Systems. Die folgenden Perspektiven zeigen Wege auf, diese zu überwinden.

7.2 Perspektiven für zukünftige Forschung

Die durchgeführte Multi-Seed-Validierung bestätigt den Kollaps als systematisches Phänomen und die Notwendigkeit von *Early Stopping*. Eine Skalierung auf größere Trainingsbatches könnte die Strategiequalität verbessern. Eine rekurrente Architektur oder ein Tiefenbild-Stapel würde dem Agenten zeitliches Gedächtnis verleihen und das *Perceptual-Aliasing*-Problem adressieren. Eine Ablationsstudie könnte den individuellen Beitrag von Tiefenbild und Zustandsvektor quantifizieren. *Domain Randomization*, insbesondere Sensorrauschen und variable Zielpositionen, könnte die Robustheit und den Transfer auf reale Hardware vorbereiten. Langfristig stellt der *Sim-to-Real*-Transfer den entscheidenden nächsten Schritt dar, um die gezeigte Leistungsfähigkeit in realen Einsatzszenarien zu validieren.

Zusammenfassung

Die autonome Landung von Drohnen in hindernisreichen Umgebungen erfordert die gleichzeitige Bewältigung von Zielnavigation und Hindernisvermeidung. Bestehende Ansätze behandeln diese Teilaufgaben überwiegend getrennt, und insbesondere in natürlichen Umgebungen mit dichter Vegetation fehlen integrierte Lösungen. Die vorliegende Arbeit untersucht, inwiefern ein *Reinforcement-Learning*-basierter Ansatz eine autonome Drohne befähigen kann, mithilfe einer minimalen Sensorkonfiguration aus Tiefenkamera und propriozeptivem Zustandsvektor in solchen Umgebungen sicher zu landen. Dazu wird ein Agent mit einer CNN-MLP-Architektur mittels *Proximal Policy Optimization* in einer simulierten Korridorumgebung mit 3D-Hindernissen im Genesis-Physiksimulator trainiert. Ein kaskadierter PID-Regler überführt die gelernten Positionsbefehle in Motorsteuerungssignale. Die Evaluation erfolgt zweiphasig: In Phase 1 erreicht der Agent in der Trainingsumgebung mit ungesesehenen Hindernissen eine Erfolgsrate von 71,0 %. In Phase 2 wird die gelernte Strategie im *Zero-Shot-Transfer* auf eine photogrammetrische Weinbergumgebung übertragen. Hier zeigt sich eine deutliche Leistungsreduktion (10,4 % Erfolgsrate bei 0,5 m Landeradius), wobei der Agent das Ziel zuverlässig anfliegt, jedoch an der terminalen Präzisionslandung scheitert. Das Training erweist sich als instabil: Nur vier von sechs Seeds konvergieren, wobei drei davon nach Erreichen des Optimums einen abrupten Strategiekollaps zeigen, was ein gezieltes *Early Stopping* erfordert. Die Ergebnisse belegen, dass RL mit minimaler Sensorik die kombinierte Aufgabe in der Trainingsumgebung löst. Der Transfer auf realistische Vegetation ist eingeschränkt möglich, wobei die Leistungsgrenzen in der Trainingsdiversität und fehlender zeitlicher Integration der Beobachtungen liegen.

Schlüsselwörter:

Reinforcement Learning, autonome Drohnenlandung, Hindernisvermeidung, Tiefensensorik, *Proximal Policy Optimization*, Genesis-Simulator

Der Quellcode sowie die trainierten Modelle sind öffentlich verfügbar unter:

https://github.com/FritzKoerner/autonomous_landing_drones_rl_bachelorthesis

Literatur

- [1] Katherine Calvin u. a. *IPCC, 2023: Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change [Core Writing Team, H. Lee and J. Romero (Eds.)]. IPCC, Geneva, Switzerland.* Intergovernmental Panel on Climate Change (IPCC), 25. Juli 2023. doi: 10.59327/IPCC/AR6-9789291691647. URL: <https://www.ipcc.ch/report/ar6/syr/> (besucht am 01.04.2026).
- [2] Andrew Hultgren, Tamma Carleton, Michael Delgado, Diana R. Gergel, Michael Greenstone, Trevor Houser, Solomon Hsiang, Amir Jina, Robert E. Kopp, Steven B. Malevich, Kelly E. McCusker, Terin Mayer, Ishan Nath, James Rising, Ashwin Rode und Jiacan Yuan. „Impacts of Climate Change on Global Agriculture Accounting for Adaptation“. In: *Nature* 642.8068 (1. Juni 2025), S. 644–652. ISSN: 1476-4687. doi: 10.1038/s41586-025-09085-w. URL: <https://doi.org/10.1038/s41586-025-09085-w> (besucht am 10.06.2026).
- [3] Tapan B. Pathak, Mahesh L. Maskey, Jeffery A. Dahlberg, Faith Kearns, Khaled M. Bali und Daniele Zaccaria. „Climate Change Trends and Impacts on California Agriculture: A Detailed Review“. In: *Agronomy* 8.25 (3 2018). ISSN: 2073-4395. doi: 10.3390/agronomy8030025. URL: <https://www.mdpi.com/2073-4395/8/3/25> (besucht am 10.06.2026).
- [4] FAO. *The future of food and agriculture: Trends and challenges*. Hrsg. von Food and Agriculture Organization of the United Nations. Rome: Food und Agriculture Organization of the United Nations, 2017. 163 S. ISBN: 978-92-5-109551-5.
- [5] United Nations. *World Population Prospects 2024: Summary of Results*. UN DESA/POP/2024/TR/NO. 9. New York, NY, USA: United Nations, 2024, S. 64. URL: https://population.un.org/wpp/assets/Files/WPP2024_Summary-of-Results.pdf (besucht am 03.05.2026).
- [6] Tom Duckett u. a. *Agricultural Robotics: The Future of Robotic Agriculture*. 2. Aug. 2018. doi: 10.48550/arXiv.1806.06762. arXiv: 1806.06762 [cs].

- URL: <http://arxiv.org/abs/1806.06762> (besucht am 03.05.2026).
Vorveröffentlichung.
- [7] Bruno Basso und John Antle. „Digital Agriculture to Design Sustainable Agricultural Systems“. In: *Nature Sustainability* 3.4 (1. Apr. 2020), S. 254–256. ISSN: 2398-9629. DOI: 10.1038/s41893-020-0510-0. URL: <https://doi.org/10.1038/s41893-020-0510-0> (besucht am 10.06.2026).
- [8] Abdul Sattar Mashori, Fuzhong Li, Muhammad Aman, Wuping Zhang, Shujie Jia, Aamir Ali, Nida Jabeen, Wangli Hao und Yuqiao Yan. „Remote Sensing through UAVs for Precision Agriculture: Applications, Technical Foundations, Current Barriers, and Future Opportunities“. In: *Smart Agricultural Technology* 14 (1. Aug. 2026), S. 102074. ISSN: 2772-3755. DOI: 10.1016/j.atech.2026.102074. URL: <https://www.sciencedirect.com/science/article/pii/S2772375526002959> (besucht am 03.05.2026).
- [9] Panagiotis Radoglou-Grammatikis, Panagiotis Sarigiannidis, Thomas Lagkas und Ioannis Moscholios. „A Compilation of UAV Applications for Precision Agriculture“. In: *Computer Networks* 172 (8. Mai 2020), S. 107148. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2020.107148. URL: <https://www.sciencedirect.com/science/article/pii/S138912862030116X> (besucht am 03.05.2026).
- [10] Sanket S. Unde, V. K. Kurkute, Sachin S. Chavan, Dadaso D. Mohite, Akshay A. Harale und Ayaan Chougale. „The Expanding Role of Multirotor UAVs in Precision Agriculture with Applications AI Integration and Future Prospects“. In: *Discover Mechanical Engineering* 4.1 (16. Sep. 2025), S. 38. ISSN: 2731-6564. DOI: 10.1007/s44245-025-00132-4. URL: <https://doi.org/10.1007/s44245-025-00132-4> (besucht am 29.01.2026).
- [11] Ridha Guebsi, Sonia Mami und Karem Chokmani. „Drones in Precision Agriculture: A Comprehensive Review of Applications, Technologies, and Challenges“. In: *Drones* 8.11 (2024), S. 686. ISSN: 2504-446X. DOI: 10.3390/drones8110686. URL: <https://www.mdpi.com/2504-446X/8/11/686> (besucht am 10.06.2026).
- [12] José Amendola, Linga Reddy Cenkeramaddi und Ajit Jha. „Drone Landing and Reinforcement Learning: State-of-Art, Challenges and Opportunities“. In: *IEEE Open*

- Journal of Intelligent Transportation Systems* 5 (2024), S. 520–539. DOI: 10.1109/OJITS.2024.3444487. URL: <https://ieeexplore.ieee.org/document/10637701> (besucht am 10.06.2026).
- [13] Aurello Patrik, Gaudi Utama, Alexander Agung Santoso Gunawan, Andry Chowanda, Jarot S. Suroso, Rizatus Shofiyanti und Widodo Budiharto. „GNSS-based Navigation Systems of Autonomous Drone for Delivering Items“. In: *Journal of Big Data* 6.1 (14. Juni 2019), S. 53. ISSN: 2196-1115. DOI: 10.1186/s40537-019-0214-3. URL: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0214-3> (besucht am 13.05.2026).
- [14] Jamie Wubben, Francisco Fabra, Carlos T. Calafate, Tomasz Krzeszowski, Johann M. Marquez-Barja, Juan-Carlos Cano und Pietro Manzoni. „Accurate Landing of Unmanned Aerial Vehicles Using Ground Pattern Recognition“. In: *Electronics* 8.12 (12. Dez. 2019). ISSN: 2079-9292. DOI: 10.3390/electronics8121532. URL: <https://www.mdpi.com/2079-9292/8/12/1532> (besucht am 14.05.2026).
- [15] Luca Bartolomei, Yves Kompis, Lucas Teixeira und Margarita Chli. „Autonomous Emergency Landing for Multicopters Using Deep Reinforcement Learning“. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Kyoto, Japan, Okt. 2022, S. 3392–3399. DOI: 10.1109/IROS47612.2022.9981152. URL: <https://ieeexplore.ieee.org/document/9981152> (besucht am 10.06.2026).
- [16] Antonio Loquercio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun und Davide Scaramuzza. „Learning High-Speed Flight in the Wild“. In: *Science Robotics* 6.59 (6. Okt. 2021), eabg5810. DOI: 10.1126/scirobotics.abg5810. eprint: <https://www.science.org/doi/pdf/10.1126/scirobotics.abg5810>. URL: <https://www.science.org/doi/10.1126/scirobotics.abg5810> (besucht am 10.06.2026).
- [17] Zhefan Xu, Xinming Han, Haoyu Shen, Hanyu Jin und Kenji Shimada. „NavRL: Learning Safe Flight in Dynamic Environments“. In: *IEEE Robotics and Automation Letters* 10.4 (Apr. 2025), S. 3668–3675. ISSN: 2377-3766. DOI: 10.1109/LRA.202

- 5.3546069. URL: <https://ieeexplore.ieee.org/document/10904341> (besucht am 10.06.2026).
- [18] Riccardo Polvara, Massimiliano Patacchiola, Marc Hanheide und Gerhard Neumann. „Sim-to-Real Quadrotor Landing via Sequential Deep Q-Networks and Domain Randomization“. In: *Robotics* 9.1 (25. Feb. 2020). ISSN: 2218-6581. DOI: 10.3390/robotics9010008. URL: <https://www.mdpi.com/2218-6581/9/1/8> (besucht am 10.06.2026).
- [19] Pawel Ladosz, Meraj Mammadov, Heejung Shin, Woojae Shin und Hyondong Oh. „Autonomous Landing on a Moving Platform Using Vision-Based Deep Reinforcement Learning“. In: *IEEE Robotics and Automation Letters* 9.5 (Mai 2024), S. 4575–4582. ISSN: 2377-3766. DOI: 10.1109/LRA.2024.3379837. URL: <https://ieeexplore.ieee.org/document/10476692/> (besucht am 14.05.2026).
- [20] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun und Davide Scaramuzza. „Champion-Level Drone Racing Using Deep Reinforcement Learning“. In: *Nature* 620.7976 (31. Aug. 2023), S. 982–987. ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4. URL: <https://doi.org/10.1038/s41586-023-06419-4> (besucht am 10.06.2026).
- [21] Victoria J. Hodge, Richard Hawkins und Rob Alexander. „Deep Reinforcement Learning for Drone Navigation Using Sensor Data“. In: *Neural Computing and Applications* 33.6 (1. März 2021), S. 2015–2033. ISSN: 1433-3058. DOI: 10.1007/s00521-020-05097-x. URL: <https://doi.org/10.1007/s00521-020-05097-x> (besucht am 10.06.2026).
- [22] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg und Demis Hassabis. „Human-Level Control through Deep Reinforcement Learning“. In: *Nature* 518.7540 (Feb. 2015), S. 529–533. ISSN: 1476-4687. DOI: 10.1038/nature14236. URL: <https://www.nature.com/articles/nature14236> (besucht am 03.05.2026).

- [23] Serhiy O. Semerikov, Pavlo P. Nechypurenko, Tetiana A. Vakaliuk, Iryna S. Mintii und Andrii O. Kolhatin. „Vision-Based Autonomous UAV Landing: A Comprehensive Review of Technologies, Techniques, and Applications“. In: *Journal of Intelligent & Robotic Systems* 111.4 (28. Okt. 2025), S. 115. ISSN: 1573-0409. DOI: 10.1007/s10846-025-02314-4. URL: <https://doi.org/10.1007/s10846-025-02314-4> (besucht am 05.05.2026).
- [24] Richard S. Sutton und Andrew Barto. *Reinforcement Learning: An Introduction*. second edition. Adaptive Computation and Machine Learning. Cambridge, Massachusetts London, England: The MIT Press, 2020. 526 S. ISBN: 9780262039246.
- [25] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford und Oleg Klimov. *Proximal Policy Optimization Algorithms*. 28. Aug. 2017. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs]. URL: <http://arxiv.org/abs/1707.06347> (besucht am 10.06.2026). Vorveröffentlichung.
- [26] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan und Philipp Moritz. „Trust Region Policy Optimization“. In: *Proceedings of the 32nd International Conference on Machine Learning*. Hrsg. von Francis Bach und David Blei. Bd. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, Juli 2015, S. 1889–1897. URL: <https://proceedings.mlr.press/v37/schulman15.html> (besucht am 10.06.2026).
- [27] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan und Pieter Abbeel. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 20. Juni 2015. DOI: 10.48550/arXiv.1506.02438. arXiv: 1506.02438 [cs.LG]. URL: <https://arxiv.org/abs/1506.02438> (besucht am 10.06.2026).
- [28] Yann LeCun, Yoshua Bengio und Geoffrey Hinton. „Deep Learning“. In: *Nature* 521.7553 (1. Mai 2015), S. 436–444. ISSN: 1476-4687. DOI: 10.1038/nature14539. URL: <https://doi.org/10.1038/nature14539> (besucht am 06.06.2026).
- [29] Benny Botsch. „Parametrische Methoden“. In: *Maschinelles Lernen - Grundlagen und Anwendungen: Mit Beispielen in Python*. Hrsg. von Benny Botsch. Berlin, Heidelberg:

- Springer, 2023, S. 61–102. ISBN: 978-3-662-67277-8. DOI: 10.1007/978-3-662-67277-8_5. URL: https://doi.org/10.1007/978-3-662-67277-8_5 (besucht am 06.06.2026).
- [30] Christoforos Kanellakis und George Nikolakopoulos. „Survey on Computer Vision for UAVs: Current Developments and Trends“. In: *Journal of Intelligent & Robotic Systems* 87.1 (1. Juli 2017), S. 141–168. ISSN: 1573-0409. DOI: 10.1007/s10846-017-0483-z. URL: <https://doi.org/10.1007/s10846-017-0483-z> (besucht am 13.05.2026).
- [31] Junqiao Wang, Zhongliang Yu, Dong Zhou, Jiaqi Shi und Runran Deng. „Vision-Based Deep Reinforcement Learning of UAV Autonomous Navigation Using Privileged Information“. In: *Drones* 8.12 (22. Dez. 2024). ISSN: 2504-446X. DOI: 10.3390/drones8120782. URL: <https://www.mdpi.com/2504-446X/8/12/782> (besucht am 10.06.2026).
- [32] Imen Jarraya, Abdulrahman Al-Batati, Muhammad Bilal Kadri, Mohamed Abdelkader, Adel Ammar, Wadii Boulila und Anis Koubaa. „Gnss-Denied Unmanned Aerial Vehicle Navigation: Analyzing Computational Complexity, Sensor Fusion, and Localization Methodologies“. In: *Satellite Navigation* 6.1 (7. Apr. 2025), S. 9. ISSN: 2662-1363. DOI: 10.1186/s43020-025-00162-z. URL: <https://doi.org/10.1186/s43020-025-00162-z> (besucht am 14.05.2026).
- [33] Lorenz Meier, Dominik Honegger und Marc Pollefeys. „PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms“. In: *2015 IEEE International Conference on Robotics and Automation (ICRA)*. Seattle, WA, USA: IEEE, Mai 2015, S. 6235–6240. ISBN: 978-1-4799-6923-4. DOI: 10.1109/ICRA.2015.7140074. URL: <http://ieeexplore.ieee.org/document/7140074/> (besucht am 14.05.2026).
- [34] Luca Tavasci, Francesco Nex und Stefano Gandolfi. „Reliability of Real-Time Kinematic (RTK) Positioning for Low-Cost Drones’ Navigation across Global Navigation Satellite System (GNSS) Critical Environments“. In: *Sensors* 24.18 (20. Sep. 2024). ISSN: 1424-8220. DOI: 10.3390/s24186096. URL: <https://www.mdpi.com/1424-8220/24/18/6096> (besucht am 14.05.2026).

- [35] Holger Voos und Haitham Bou-Ammar. „Nonlinear Tracking and Landing Controller for Quadrotor Aerial Robots“. In: *2010 IEEE International Conference on Control Applications*. Yokohama, Japan: IEEE, Sep. 2010, S. 2136–2141. ISBN: 978-1-4244-5362-7. DOI: 10.1109/CCA.2010.5611204. URL: <http://ieeexplore.ieee.org/document/5611204/> (besucht am 13.05.2026).
- [36] Tao Yang, Peiqi Li, Huiming Zhang, Jing Li und Zhi Li. „Monocular Vision SLAM-Based UAV Autonomous Landing in Emergencies and Unknown Environments“. In: *Electronics* 7.5 (15. Mai 2018). ISSN: 2079-9292. DOI: 10.3390/electronics7050073. URL: <https://www.mdpi.com/2079-9292/7/5/73> (besucht am 07.02.2026).
- [37] Robinroy Peter, Lavanya Ratnabala, Demetros Aschu, Aleksey Fedoseev und Dzmitry Tsetserukou. „TornadoDrone: Bio-inspired DRL-based Drone Landing on 6D Platform with Wind Force Disturbances“. In: *Proceedings of the 2024 IEEE International Conference on Robotics and Biomimetics (ROBIO)* (Bangkok, Thailand). Dez. 2024, S. 516–521. DOI: 10.1109/ROBIO64047.2024.10907520. URL: <https://ieeexplore.ieee.org/document/10907520/> (besucht am 07.03.2026).
- [38] Minwoo Kim, Jongyun Kim, Minjae Jung und Hyondong Oh. „Towards monocular vision-based autonomous flight through deep reinforcement learning“. In: *Expert Systems with Applications* 198 (15. Juli 2022), S. 116742. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2022.116742. URL: <https://www.sciencedirect.com/science/article/pii/S0957417422002111> (besucht am 16.05.2026).
- [39] Peng Wei, Prabhath Ragbir, Stavros G. Vougioukas und Zhaodan Kong. „Vision-based navigation of unmanned aerial vehicles in orchards: An imitation learning approach“. In: *Computers and Electronics in Agriculture* 238 (1. Nov. 2025), S. 110802. ISSN: 0168-1699. DOI: 10.1016/j.compag.2025.110802. URL: <https://www.sciencedirect.com/science/article/pii/S0168169925009081> (besucht am 03.06.2026).
- [40] Mauro Martini, Simone Cerrato, Francesco Salvetti, Simone Angarano und Marcello Chiaberge. „Position-Agnostic Autonomous Navigation in Vineyards with Deep Reinforcement Learning“. In: *2022 IEEE 18th International Conference on Automation*

- Science and Engineering (CASE)*. Mexico City, Mexico, Aug. 2022, S. 477–484. DOI: 10.1109/CASE49997.2022.9926582. URL: <https://ieeexplore.ieee.org/abstract/document/9926582> (besucht am 03.06.2026).
- [41] Julian Eßer, Gabriel B. Margolis, Oliver Urbann, Sören Kerner und Pulkit Agrawal. „Action Space Design in Reinforcement Learning for Robot Motor Skills“. In: *Proceedings of The 8th Conference on Robot Learning*. Hrsg. von Pulkit Agrawal, Oliver Kroemer und Wolfram Burgard. PMLR, 12. Jan. 2025, S. 4021–4032. URL: <https://proceedings.mlr.press/v270/esser25a.html> (besucht am 10.06.2026).
- [42] Elia Kaufmann, Leonard Bauersfeld und Davide Scaramuzza. „A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight“. In: *2022 International Conference on Robotics and Automation (ICRA)*. Philadelphia, PA, USA: IEEE, 23. Mai 2022, S. 10504–10510. ISBN: 978-1-7281-9681-7. DOI: 10.1109/ICRA46639.2022.9811564. URL: <https://ieeexplore.ieee.org/document/9811564/> (besucht am 10.06.2026).
- [43] Nikita Rudin, David Hoeller, Philipp Reist und Marco Hutter. „Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning“. In: *Proceedings of the 5th Conference on Robot Learning*. Hrsg. von Aleksandra Faust, David Hsu und Gerhard Neumann. Bd. 164. Proceedings of Machine Learning Research. PMLR, Nov. 2022, S. 91–100. URL: <https://proceedings.mlr.press/v164/rudin22a.html> (besucht am 10.06.2026).
- [44] Marco S. Tayar, Lucas K. de Oliveira, Felipe Andrade G. Tommaselli, Juliano D. Negri, Thiago H. Segreto, Ricardo V. Godoy und Marcelo Becker. „Autonomous UAV Flight Navigation in Confined Spaces: A Reinforcement Learning Approach“. In: *2025 Latin American Robotics Symposium (LARS)*. Monterrey, Mexico, Okt. 2025, S. 1–6. DOI: 10.1109/LARS69345.2025.11273007. URL: <https://ieeexplore.ieee.org/document/11273007> (besucht am 10.06.2026).
- [45] Andrew Y. Ng, Daishi Harada und Stuart J. Russell. „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping“. In: *Proceedings of the Sixteenth International Conference on Machine Learning*. ICML '99. San Francis-

- co, CA, USA: Morgan Kaufmann Publishers Inc., 27. Juni 1999, S. 278–287. ISBN: 1558606122.
- [46] Karl W Cobbe, Jacob Hilton, Oleg Klimov und John Schulman. „Phasic Policy Gradient“. In: *Proceedings of the 38th International Conference on Machine Learning*. Hrsg. von Marina Meila und Tong Zhang. Bd. 139. Proceedings of Machine Learning Research. PMLR, Juli 2021, S. 2020–2027. URL: <https://proceedings.mlr.press/v139/cobbe21a.html> (besucht am 10.06.2026).
- [47] Clemens Schwarke, Mayank Mittal, Nikita Rudin, David Hoeller und Marco Hutter. *RSL-RL: A Learning Library for Robotics Research*. Version 1. 13. Sep. 2025. DOI: 10.48550/arXiv.2509.10771. arXiv: 2509.10771 [cs.RO]. URL: <http://arxiv.org/abs/2509.10771> (besucht am 10.06.2026). Vorveröffentlichung.
- [48] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel und Sergey Levine. „Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor“. In: *Proceedings of the 35th International Conference on Machine Learning*. Hrsg. von Jennifer Dy und Andreas Krause. Bd. 80. Proceedings of Machine Learning Research. PMLR, 3. Juli 2018, S. 1861–1870. URL: <https://proceedings.mlr.press/v80/haarnoja18b.html> (besucht am 10.06.2026).
- [49] Genesis Authors. *Genesis: A Generative and Universal Physics Engine for Robotics and Beyond*. Dez. 2024. URL: <https://github.com/Genesis-Embodied-AI/Genesis> (besucht am 20.05.2026).
- [50] Nathan Koenig und Andrew Howard. „Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator“. In: *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*. Bd. 3. Sendai, Japan, Sep. 2004, S. 2149–2154. ISBN: 0-7803-8463-6. DOI: 10.1109/IROS.2004.1389727. URL: <https://ieeexplore.ieee.org/document/1389727> (besucht am 10.06.2026).
- [51] Jacopo Panerati, Hehui Zheng, SiQi Zhou, James Xu, Amanda Prorok und Angela P. Schoellig. *Learning to Fly – a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control*. 25. Juli 2021. DOI: 10.48550/a

- rxiv.2103.02142. arXiv: 2103.02142 [cs.RO]. URL: <http://arxiv.org/abs/2103.02142> (besucht am 10.06.2026). Vorveröffentlichung.
- [52] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa und Gvriiel State. „Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning“. In: *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. 2021. URL: https://openreview.net/forum?id=fgFBtYgJQX_ (besucht am 10.06.2026).
- [53] Laura Downs, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B. McHugh und Vincent Vanhoucke. „Google Scanned Objects: A High-Quality Dataset of 3D Scanned Household Items“. In: *2022 International Conference on Robotics and Automation (ICRA)*. Philadelphia, PA, USA: IEEE Press, 23. Mai 2022, S. 2553–2560. DOI: 10.1109/ICRA46639.2022.9811809. URL: <https://doi.org/10.1109/ICRA46639.2022.9811809> (besucht am 10.06.2026).
- [54] Bradley Efron und R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman and Hall/CRC, 15. Mai 1994. 456 S. ISBN: 978-0-429-24659-3. DOI: 10.1201/9780429246593.
- [55] Skander Moalla, Andrea Miele, Daniil Pyatko, Razvan Pascanu und Caglar Gulcehre. „No Representation, No Trust: Connecting Representation, Collapse, and Trust Issues in PPO“. In: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Hrsg. von A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak und C. Zhang. Bd. 37. Curran Associates, Inc., 2024, S. 69652–69699. DOI: 10.52202/079017-2226. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/81166fbd9cc5adf14031cdb69d3fd6a8-Paper-Conference.pdf (besucht am 10.06.2026).
- [56] Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon und Aaron Courville. „The Primacy Bias in Deep Reinforcement Learning“. In: *Proceedings of the 39th International Conference on Machine Learning (ICML)*. Hrsg. von Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu und Sivan Sabato.

- Bd. 162. Proceedings of Machine Learning Research. PMLR, Juli 2022, S. 16828–16847. URL: <https://proceedings.mlr.press/v162/nikishin22a.html> (besucht am 10.06.2026).
- [57] Jette Randløv und Preben Alstrøm. „Learning to Drive a Bicycle Using Reinforcement Learning and Shaping“. In: *Proceedings of the Fifteenth International Conference on Machine Learning*. ICML '98. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 24. Juli 1998, S. 463–471. ISBN: 1558605568.
- [58] Steven D. Whitehead und Dana H. Ballard. „Learning to Perceive and Act by Trial and Error“. In: *Machine Learning* 7.1 (1. Juli 1991), S. 45–83. ISSN: 1573-0565. DOI: 10.1023/A:1022619109594. URL: <https://doi.org/10.1023/A:1022619109594> (besucht am 30.05.2026).
- [59] Abhik Singla, Sindhu Padakandla und Shalabh Bhatnagar. „Memory-Based Deep Reinforcement Learning for Obstacle Avoidance in UAV With Limited Environment Knowledge“. In: *IEEE Transactions on Intelligent Transportation Systems* 22.1 (Jan. 2021), S. 107–118. ISSN: 1558-0016. DOI: 10.1109/TITS.2019.2954952. URL: <https://ieeexplore.ieee.org/document/8917687> (besucht am 10.06.2026).

Anhang

Anhangsverzeichnis

Anhang A	Ergänzende Trainingsverläufe Seed 0	X
Anhang B	Standardabweichung über alle Seeds	XII
Anhang C	Phase-1-Evaluationsmetriken	XIII
Anhang D	Phase-1-Evaluationsmetriken (oberflächenbasierte Kollisionserkennung)	XIV
Anhang E	Phase-1-Evaluationsmetriken (RRT-Connect)	XV
Anhang F	Phase-2-Evaluationsmetriken	XVI
Anhang G	Phase-2-Evaluationsmetriken (RRT-Connect)	XVII
Anhang H	Quellcode-Repository	XVIII
Anhang I	PID-Reglerparameter	XIX

A Ergänzende Trainingsverläufe Seed 0

Alle Verläufe sind mit exponentiellem gleitendem Mittelwert (EMA, $\alpha = 0,9$) geglättet.

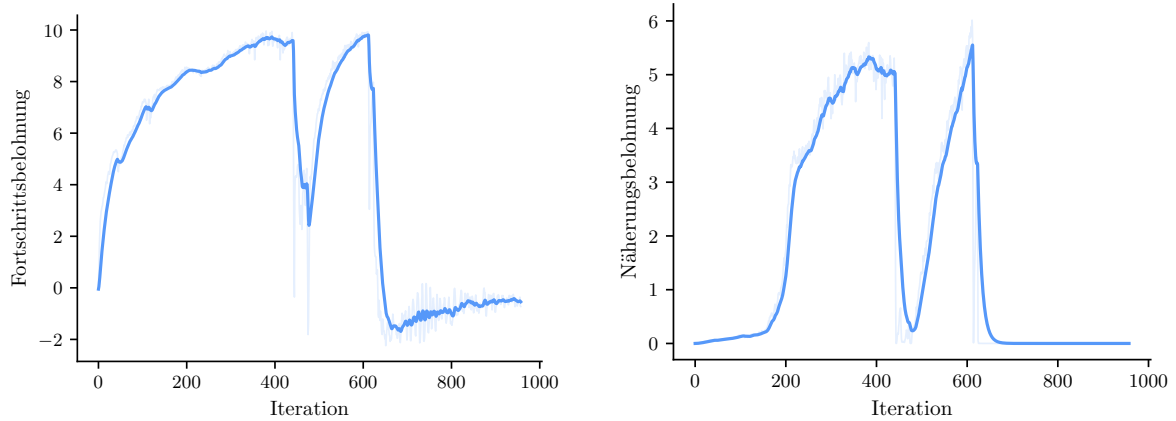


Abbildung A1.: Fortschrittsbelohnung (links) und Näherungsbelohnung (rechts). Beide Komponenten steigen mit zunehmender Annäherung an das Ziel.

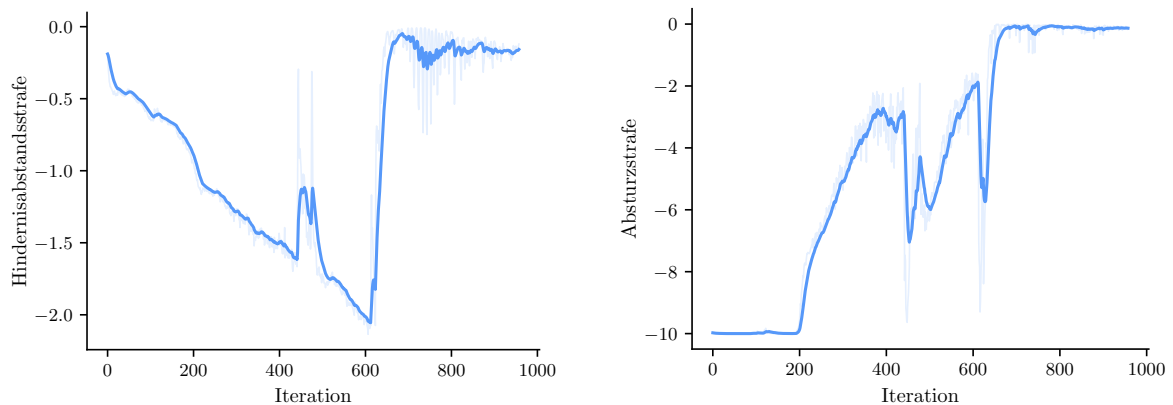


Abbildung A2.: Hindernisabstandsstrafe (links) und Absturzstrafe (rechts). Beide Strafkomponten fallen mit zunehmender Kollisionsvermeidung.

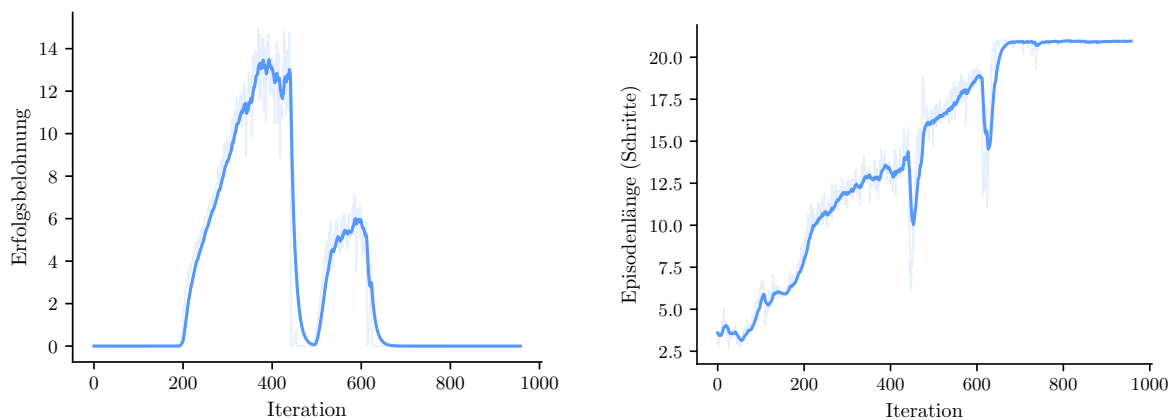


Abbildung A3.: Erfolgsbelohnung (links) und mittlere Episodenlänge (rechts). Der Anstieg der Erfolgsbelohnung markiert den Beginn erfolgreicher Landungen. Die Episodenlänge gibt die Anzahl der Entscheidungsschritte pro Episode an.

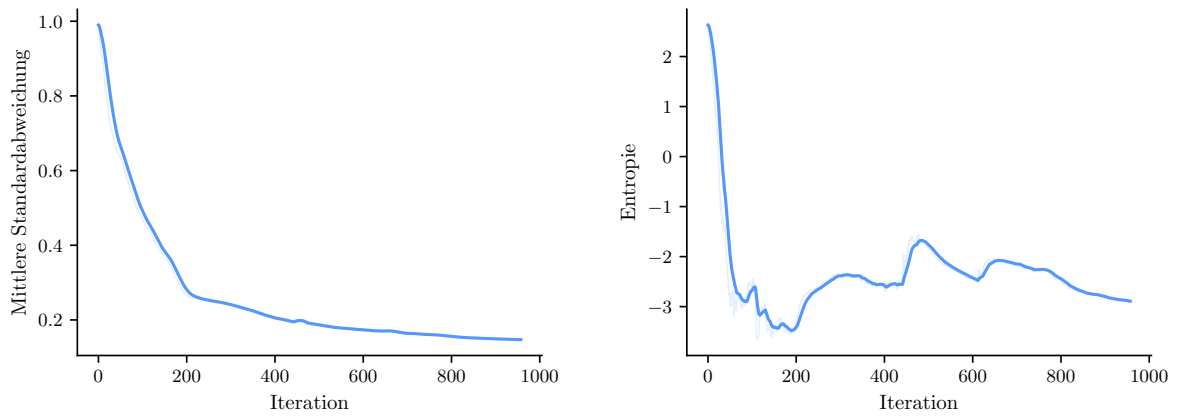


Abbildung A4.: Mittlere Standardabweichung der Aktionsverteilung (links) und Entropie (rechts). Beide Maße fallen monoton und spiegeln die abnehmende Exploration wider.

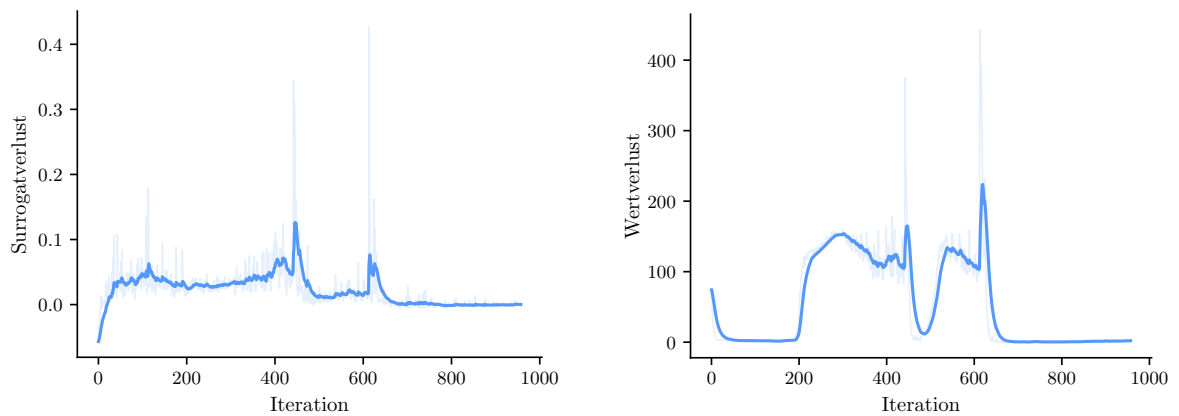


Abbildung A5.: PPO-Surrogatverlust (links) und Wertverlust (rechts).

B Standardabweichung über alle Seeds

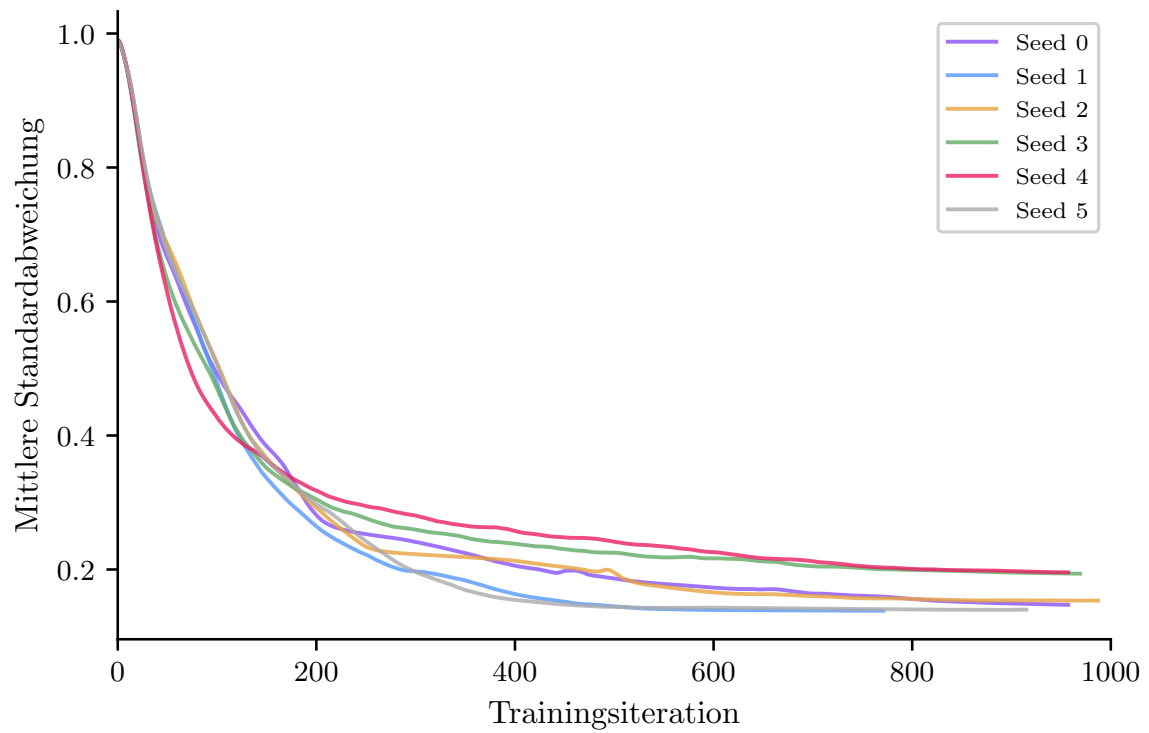


Abbildung B1.: Mittlere Standardabweichung der Aktionsverteilung über alle sechs Seeds. Der Verlauf ist nahezu identisch und fällt von 0,99 auf circa 0,15, unabhängig vom jeweiligen Belohnungsverlauf. Geglättet mit EMA $\alpha = 0,9$.

C Phase-1-Evaluationsmetriken

Tabelle C1.: Phase-1-Ergebnisse des RL-Agenten (1 000 Episoden, Checkpoint 400, ungesehene Hindernisse). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode, $B = 10\,000$). Landezeit und Pfadeffizienz beziehen sich ausschließlich auf erfolgreiche Episoden ($n = 710$).

Metrik	Wert	95 %-KI / IQR
Erfolgsrate	71,0 %	[68,2; 73,8]
Hinderniskollision	28,8 % (288)	[26,0; 31,6]
Verlassen des Korridors	0,2 % (2)	[0,0; 0,5]
Bodenkollision	0,0 % (0)	—
Timeout	0,0 % (0)	—
Landezeit (Median)	30,0 s	IQR [27,0; 36,0]
Pfadeffizienz (Median)	1,19	IQR [1,13; 1,25]
Min. Hindernisabstand (Mittel)	0,52 m	[0,50; 0,54]

D Phase-1-Evaluationsmetriken (oberflächenbasierte Kollisionserkennung)

Tabelle D1.: Phase-1-Ergebnisse des RL-Agenten mit oberflächenbasierter Kollisionserkennung (1 000 Episoden, Checkpoint 400, ungesehene Hindernisse). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode, $B = 10\,000$). Landezeit und Pfadeffizienz beziehen sich ausschließlich auf erfolgreiche Episoden ($n = 984$).

Metrik	Wert	95 %-KI / IQR
Erfolgsrate	98,4 %	[97,6; 99,1]
Hinderniskollision	1,2 % (12)	[0,6; 1,9]
Verlassen des Korridors	0,2 % (2)	[0,0; 0,5]
Bodenkollision	0,0 % (0)	—
Timeout	0,2 % (2)	[0,0; 0,5]
Landezeit (Median)	33,0 s	IQR [30,0; 36,0]
Pfadeffizienz (Median)	1,19	IQR [1,13; 1,25]
Min. Hindernisabstand (Mittel)	0,51 m	[0,49; 0,53]

E Phase-1-Evaluationsmetriken (RRT-Connect)

Tabelle E1.: Phase-1-Ergebnisse des RRT-Connect-Planers (1 000 Episoden, Checkpoint 400, ungesehene Hindernisse). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode, $B = 10\,000$). Landezeit und Pfadefizienz beziehen sich ausschließlich auf erfolgreiche Episoden ($n = 820$).

Metrik	Wert	95 %-KI / IQR
Erfolgsrate	82,0 %	[79,6; 84,4]
Hinderniskollision	18,0 % (180)	[15,6; 20,4]
Verlassen des Korridors	0,0 % (0)	—
Bodenkollision	0,0 % (0)	—
Timeout	0,0 % (0)	—
Landezeit (Median)	42,0 s	IQR [42,0; 48,0]
Pfadefizienz (Median)	1,09	IQR [1,07; 1,20]
Min. Hindernisabstand (Mittel)	0,48 m	[0,47; 0,49]

F Phase-2-Evaluationsmetriken

Tabelle F1.: Phase-2-Ergebnisse des RL-Agenten (1 000 Episoden, Checkpoint 400, Weinberg-Transfer, Erfolgsradius 0,5 m). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode, $B = 10\,000$). Landezeit und Pfadeffizienz beziehen sich ausschließlich auf erfolgreiche Episoden ($n = 104$).

Metrik	Wert	95 %-KI / IQR
Erfolgsrate	10,4 %	[8,5; 12,4]
Terrain-Kollision	15,9 % (159)	[13,6; 18,2]
Timeout	73,7 % (737)	[71,0; 76,4]
Bodenkollision	entfällt (kein fester Bodenkollisions-Check)	
Verlassen des Korridors	entfällt (kein Korridorbegrenzungsrahmen)	
Landezeit (Median)	36,0 s	IQR [33,0; 36,0]
Pfadeffizienz (Median)	1,07	IQR [1,05; 1,09]
Finale Zieldistanz (Median)	1,19 m	—
Min. Terrain-Abstand (Mittel)	1,02 m	[0,98; 1,06]

G Phase-2-Evaluationsmetriken (RRT-Connect)

Tabelle G1.: Phase-2-Ergebnisse des RRT-Connect-Planers (1 000 Episoden, Checkpoint 400, Weinberg-Transfer, Erfolgswert 0,5 m). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode, $B = 10\,000$). Landezeit und Pfadefizienz beziehen sich ausschließlich auf erfolgreiche Episoden ($n = 857$).

Metrik	Wert	95 %-KI / IQR
Erfolgsrate	85,7 %	[83,5; 87,9]
Terrain-Kollision	7,1 % (71)	[5,6; 8,7]
Timeout	7,9 % (79)	[6,3; 9,6]
Bodenkollision	entfällt (kein fester Bodenkollisions-Check)	
Verlassen des Korridors	entfällt (kein Korridorbegrenzungsrahmen)	
Landezeit (Median)	42,0 s	IQR [39,0; 48,0]
Pfadefizienz (Median)	0,99	IQR [0,97; 1,00]
Min. Terrain-Abstand (Mittel)	0,46 m	[0,45; 0,47]

H Quellcode-Repository

Der vollständige Quellcode, die trainierten Modelle, Evaluationsergebnisse sowie die Skripte zur Reproduktion aller Experimente sind öffentlich verfügbar unter:

```
https://github.com/FritzKoerner/autonomous\_landing\_drones\_rl\_bachelorthesis
```

Das Repository enthält die Simulationsumgebungen, den kaskadierenden PID-Regler, die Trainings- und Evaluationsskripte, das finale Modell (Checkpoint 400) mit allen Evaluationsergebnissen sowie die Kontrollläufe (5 Seeds) zur statistischen Varianzanalyse.

I PID-Reglerparameter

Tabelle I1.: PID-Verstärkungen des kaskadierenden Reglers.

Regelkreis	K_p	K_i	K_d
Position x/y	1,0	0	0,7
Position z	1,5	0	1,0
Geschwindigkeit x/y	16,0	0	8,0
Geschwindigkeit z	100,0	2,0	10,0
Roll / Nick	6,0	0	3,0
Gier	0,5	0	0,8

Offenlegung der Benutzung von generativer KI

Bei der Bearbeitung von Abschlussarbeiten ist KI-Nutzung ausdrücklich erlaubt und wird neutral bewertet. Studierende müssen KI-Verwendung „wissenschaftlich angemessen“ offenlegen, einschließlich verwendeter Modelle, Zweck und Umfang der Nutzung.

Anwendungsbereich	Nutzung
Kreative Arbeit (z. B. Generierung neuer Ideen)	Ja – Claude Opus 4. Diskussion der Forschungsfragen und Evaluationsergebnisse.
Recherche (z. B. Auffinden neuer Quellen)	Ja – Claude Code mit Websuche. Identifikation relevanter Fachartikel im Bereich drohnenbasierter Landung und Hindernisvermeidung.
Schreibprozess (z. B. Generierung von Textpassagen)	Ja – Claude Code. Formulierungshilfen in deutscher Fachsprache. Alle generierten Textpassagen wurden manuell überprüft und überarbeitet.
Text-Analyse, Literatursynthese oder Datenaufbereitung	Ja – Claude Code. Erstellung von Python-Plotskripten, Fehlerbehebung im Programmcode und Code-Optimierung.
Übersetzungen	Nein.
Generieren von Bildmaterial (z. B. für Grafiken und Diagramme)	Nein.
Begutachtung der eigenen Arbeit	Nein.

Erklärung

Ich versichere, dass ich die vorliegende Arbeit mit dem Thema:

*„Autonomes Landen von Drohnen in Agrarlandschaften basierend auf
Deep-Reinforcement-Learning“*

selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Leipzig, den 15. Juni 2026

FRITZ KÖRNER